OCR Water Meter Reader: Low cost and power Vision based OCR solution for reading water meters.

Braven Chiam
NYP Electronic and Computer Engineering (242350F)
Website: www.chiambucket.com
Email: braven@chiambucket.com
Read the article on this project: www.chiambucket.com/waterslop

AI Use

AI tools were used heavily in the development of this project. (read the Agentic Workflow section)

However, most of this report was written by me in my own words, except for the introduction section and certain graphs. Other forms of AI use in this report include research, chart generation and data verification.



This project scores an overall* Level 3 in my Scale of AI use chart

Visit www.chiambucket.com/ai-use For more info

*Certain sections may have differing scores from the project overall score.

Abstract

Research and develop a fully integrated low energy and cost, battery powered OCR water meter reader solution, first by developing a 693K parameter NN OCR pipeline that detects and reads the digits on a meter. The model, DrumNet V10 runs entirely on the ESP32-S3 MCU of the reader, it achieves a reading accuracy of 56%, with plans to improve. Readings take about 2 seconds, including sending the readings to a database through LoRaWAN. The targeted lifecycle of this meter is 2 years at 4 readings made a day.

Objective

Developing a LoRaWAN enabled Vision OCR water meter reader that is battery powered (lasting up to 2 years), low-cost and maintenance. While targeting a 99% reading accuracy.

Acknowledgements

I am deeply grateful to the many individuals who have contributed to the progress of this project.

First and foremost, I would like to express my sincere gratitude to *Mr Alex Oh (NYP)* for his invaluable guidance, expertise, and encouragement through this endeavor. His insights and advice have been instrumental in shaping the direction and outcome of this work.

I am very grateful for *Mr Soh (NYP)* for his assistance and guidance. Without which, I would not have been able to get the camera working.

I also wish to thank *Mr Lee Chang Fatt (NYP)* for providing me with a LoRaWAN gateway for testing.

Contents

1. Introduction 5
1.1 Background 5
1.2 Methodology and Design 6
2. Software 7
2.1 MeterSim 7
2.2 SimGen2 11
2.3 Agentic Workflow and Model Training 13
2.4 DrumNet Neural Network Architecture 16
2.4.1 The Problem 16
2.4.2 Network Architecture 16
2.4.3 WindowNet 18
2.4.4 DrumNet Reader Ensemble 20
2.4.5 DrumNet V10 Summary 23
2.5 Firmware (Agentic Workflow II) 23
2.5.1 Reader Firmware 24
2.5.2 Dashboard 26
2.5.3 LoRaWAN and BLE Payloads 26
3. Hardware 28
3.1 Processor (MCU) 28
3.2 Battery Technology 30
3.3 Power Optimization 31
3.3.1 Software-Based Optimizations 31
3.3.2 Hardware-Based Optimizations 33
3.4 Camera Choice 34
3.5 Prototype V1 38
3.6 Prototype V2 39
3.6.1 Planning and Layout 39
3.6.2 Power Circuit 40
3.6.3 Peripherals and MCU 41
3.6.4 PCB Layout and Manufacturing 42
3.6.5 PCB Assembly 43
3.6.6 Interior Enclosure 48
3.6.7 Exterior Enclosure (Meter Adapter) 50
3.6.8 Camera Calibration 52
3.6.9 Setup and Configuration (ConfigKing) 55
4. Conclusion 56
4.1 Energy Use and Longevity 56
4.2 WindowNet and Accuracy 56
4.3 Shell and Mounting 56
4.4 Closing Remarks 56
References 57

List of Figures

Figure 1.1 – Typical water meters in Singapore: brass meter (left) and Aquadis+ meter (right) 5
Figure 1.2 – How a reading travels from the meter to the dashboard 6
Figure 2.1 – Prototype V0.5 (ESP32-EYE) pointed at the MeterSim display 7
Figure 2.2 – MeterSim control interface 7
Figure 2.3 – The nine meter designs available in MeterSim (rendered by MeterSim) 8
Figure 2.4 – Simulating half-rolled drums in MeterSim 8
Figure 2.5 – Surface and weather effects available in MeterSim 9
Figure 2.6 – Preset conditions available in MeterSim 10
Figure 2.7 – MeterSim display and control panel with the ESP32-EYE reader 10
Figure 2.8 – The processing stages of SimGen2 11
Figure 2.9 – Real capture (magenta) overlaid on its SimGen2 synthetic image (green); uncoloured areas match 11
Figure 2.10 – Real and synthetic image pairs (left: real, right: SimGen2) 12
Figure 2.11 – The self-improvement loop followed by each training agent 13
Figure 2.12 – The agents, the shared resources and the scheduler 14
Figure 2.13 – Scheduler bookings for the compute resources (top) and the Jetson rig (bottom) 14
Figure 2.14 – How T3 Code and Tailscale connect the agents, machines and test rig 15
Figure 2.15 – A half-rolled drum: the last drum is caught between two digits 16
Figure 2.16 – An early version of WindowNet's detection and cutting results 16
Figure 2.17 – DrumNet V10 architecture overview 17
Figure 2.18 – The Spotter network (DetNet) 18
Figure 2.19 – The Refiner network (RefNet) 19
Figure 2.20 – Cutting the region of interest into individual drums (ROI drawn by hand) 20
Figure 2.21 – A single DrumNet reader network 21
Figure 2.22 – Ensemble vote, carry rule and final reading 22
Figure 2.23 – Firmware development setup made available to the agents 24
Figure 2.24 – Reader firmware flowchart for a typical deployment 25
Figure 2.25 – Reader monitor dashboard served by the Jetson Nano 26
Figure 2.26 – Reading payload (LoRaWAN port 10) 26
Figure 2.27 – Radio test payload (LoRaWAN port 11) 27
Figure 2.28 – DrumNet recovery event payload (LoRaWAN port 12) 27
Figure 2.29 – BLE protocol between ConfigKing and the reader 27
Figure 3.1 – SVGA and CIF sub-sampling modes of the OV2640 (OmniVision Technologies, 2006) 31
Figure 3.2 – Comparison of OV2640 capture modes: crop of the drums (top), full frame (bottom) and capture time 32
Figure 3.3 – Banding vs no banding when the LEDs are PWM-dimmed 32
Figure 3.4 – In-camera greyscale vs colour capture 32
Figure 3.5 – Focus charts: Chart A (left, hand-drawn) and Chart B (right; Westin, 2010) 34
Figure 3.6 – Focus test setup (left) and the camera web applet (right) 34
Figure 3.7 – Focus test results for the OV2640 with the 160° FOV lens 35
Figure 3.8 – Focus test results for the OV2640 with the 120° FOV lens 36
Figure 3.9 – Chart B at 4 cm: 160° lens (left) and 120° lens (right) 37
Figure 3.10 – Focus tapering effect of the 160° lens 37
Figure 3.11 – Prototype V1 block diagram 38
Figure 3.12 – Prototype V1 assembled 38
Figure 3.13 – Power circuits: schematic (left) and location on the board (right) 40
Figure 3.14 – MCU and peripheral circuits: schematic (left) and location on the board (right) 41
Figure 3.15 – Full board schematic from KiCad 42

Figure 3.16 – PCB layout and 3D renders of the board from KiCad 42
Figure 3.17 – Prototype V2 PCB next to an ESP32-S3 development board 43
Figure 3.18 – The solder stencil (left) and the board with solder paste applied (right) 43
Figure 3.19 – Components laid out for assembly 44
Figure 3.20 – The assembled board on the benchtop power supply 44
Figure 3.21 – Microscope views of the 3.3 V buck-boost chip 45
Figure 3.22 – 3.3 V bypass using the regulator of a scrapped ESP32 board 46
Figure 3.23 – Mirrored camera pin-out (left) and the flipped camera (right) 47
Figure 3.24 – PCLK test point (left) and PCLK pin at the camera connector (right) 47
Figure 3.25 – First successful image capture on Prototype V2 48
Figure 3.26 – KiCad 3D model used as the reference for the enclosure design 48
Figure 3.27 – Notable features of the interior enclosure 49
Figure 3.28 – Final interior enclosure (left) and earlier revisions (right) 49
Figure 3.29 – Meter adapter: top view (left) and bottom view (right) 50
Figure 3.30 – Distance from the camera lens to the meter face (about 4 cm) 50
Figure 3.31 – Cuts made so the adapter sits flush against the meter 51
Figure 3.32 – The completed adapter 51
Figure 3.33 – Reader setup procedure using ConfigKing 55

List of Tables

Table 2.1 – Key figures for the on-board reading pipeline (WindowNet + DrumNet V10-s10) ..... **Error! Bookmark not defined.**
Table 3.1 – Unit price of the ESP32-S3R8 vs the Alif Semiconductor Ensemble E3 by order quantity (USD) 28
Table 3.2 – Comparison of the Espressif ESP32-S31, ESP32-S3 and ESP32-P4 29
Table 3.3 – Comparison of primary battery chemistries used in IoT devices 30
Table 3.4 – Camera settings and their observed image effects 52

1. Introduction

1.1 Background

Singapore has about 1.6 million water meters, and they've been read by hand once every two months, with the bill for the month in between estimated from earlier usage (Poon, n.d.). Six readings a year per meter is enough for billing. It's far too coarse to catch a leak early or to show a household how its habits change what it uses.

PUB's answer is the Smart Water Meter Programme (PUB, n.d.-b). In 2021 it appointed SP Services to install the first 300,000 smart meters across homes, businesses and industrial premises, with installation due to finish in 2023 (PUB, 2021). These meters send readings to PUB every day over an Itron network, and customers track their usage and get leak alerts through an online portal (PUB, 2021; Smart Water Magazine, 2022). The rollout was slowed by the global microchip shortage, and PUB has said it will review the first phase before going further, with no date set for the rest of the island (Ministry of Sustainability and the Environment [MSE], 2022). So for more than a million meters, the two-month cycle carries on. Digitizing each of them means a plumber, a new meter and a network to carry its data.

There's a cheaper route. The register on a mechanical meter already shows the reading, so a camera fixed over it can read the same digits a person does. The best-known open-source example is AI-on-the-edge-device, which its developers say costs under €10 on an ESP32-CAM (jomjol, n.d.). It photographs the meter at set intervals and lines the image up against reference marks. It then crops out the digit regions and reads each one with TensorFlow Lite models on the board, and publishes the result over MQTT or a REST API (jomjol, n.d.). That proves the hardware is cheap enough. Its documentation doesn't report accuracy figures, though, and each user draws the digit regions by hand during setup. How well it reads depends on how each installation was set up.

Reading a meter by camera is harder than it looks. The register sits behind glass that catches glare and collects dirt, and the meter is often in a dim cabinet or in shadow. The drums turn continuously, so the last one is often caught between two digits, where either answer can be argued. Singapore's household meters aren't all one design either. PUB's own reading guide shows a compact brass meter with four black and four red drums, next to a white Aquadis+ face with five black and three red (PUB, n.d.-a), as shown in Figure 1.1.

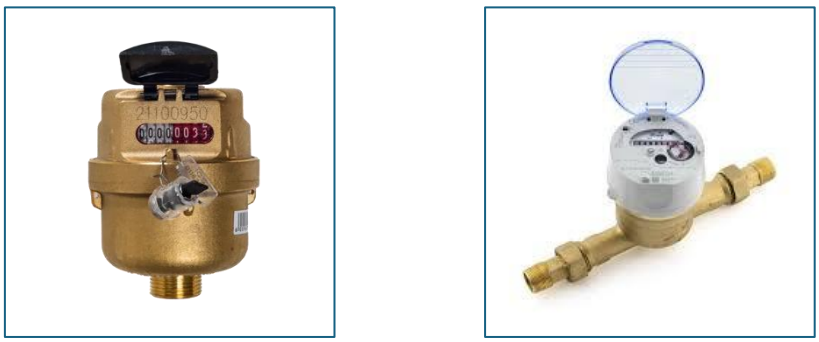


Figure 1.1 – Typical water meters in Singapore: brass meter (left) and Aquadis+ meter (right)

This project builds a low-cost sensor that mounts over an existing mechanical meter and reports its reading as text, with no plumbing work. It starts from an existing AI image node reference design and replaces that design's generic image classification with a meter-reading pipeline. The pipeline runs entirely on an ESP32-S3 in three stages. A detector finds the drum window, a refiner corrects the window's edges, and a reader reads each drum, including digits caught halfway through a turn.

Production readers send their readings over LoRaWAN; Wi-Fi is only used by the reader on the training rig. The target is 99% accuracy per full reading under outdoor conditions, measured against known correct readings.

1.2 Methodology and Design

I listed the most important design specifications that the reader would be designed around:

1. **Ultra-Low Power**
   The meter needs to be able to run on batteries while being able to last for 2 years.
2. **Low Cost**
   The meter should target a cost of < S$100 per piece, not inclusive of installation labor.
3. **High reliability**
   The meter should target ≥ 99% meter accuracy (the whole reading correct) and ≥ 99.9% digit accuracy (each drum correct).
4. **Easy deployment**
   The meter should be easy to mount and configure, with minimal calibration
5. **Modularity**
   The meter should be easily repairable and upgradable.
6. **Secure and safe**
   The meter should be able to resist tampering. As its readings will directly affect billing.
7. **Weather sealed**
   The meter should be weather sealed to avoid weather changes from causing missed readings.

The ESP32-S3 with 8MB of PSRAM was selected as the microcontroller, and the OV2640 with a 160° FOV was selected as the camera. The camera will be pointed at the water meter and will take readings on 6-hour intervals. After the reading is made on device, the numerical reading is transmitted over LoRaWAN to a database storing the readings, the readings can be accessed by the user and the utility provider over a dashboard for billing and monitoring purposes. Note that the original image is never transmitted; only the predicted reading is. Figure 1.2 shows how data travels through the system.

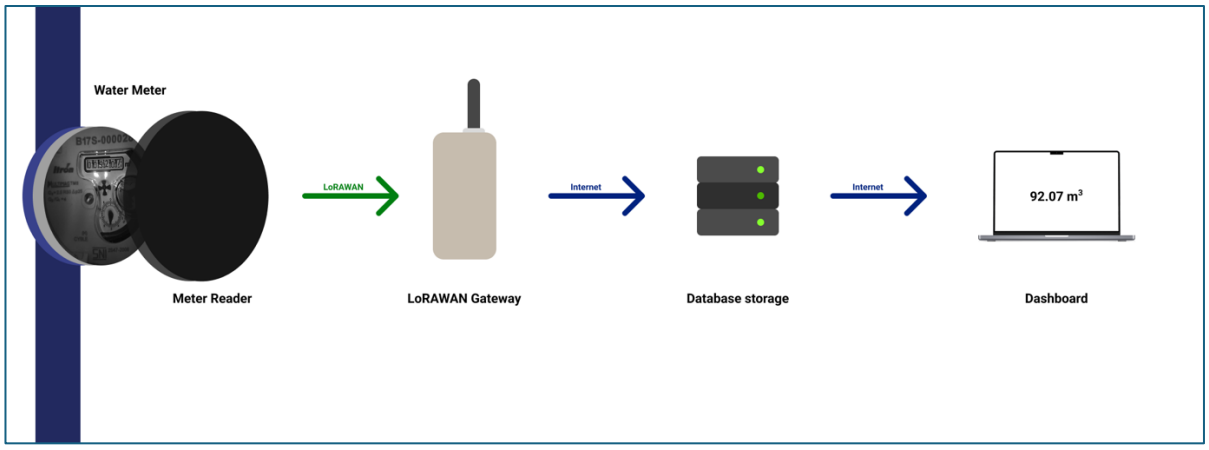


Figure 1.2 – How a reading travels from the meter to the dashboard

2. Software

2.1 MeterSim

The first version of MeterSim was built alongside Prototype V0.5's firmware, they work in conjunction to allow Prototype V0.5 to automatically be controlled by MeterSim so labeled image data could be generated for model training. All the software and firmware builds were entirely written by Claude. Figure 2.1 shows the setup.



Figure 2.1 – Prototype V0.5 (ESP32-EYE) pointed at the MeterSim display

MeterSim is built to be used by agentic AI agents instead of humans, so the control panel serves as a monitoring platform for humans to see the progress made by the agents. Agents use MeterSim as a framework and platform to build tests, collect data and verify accuracy during the process of creating DrumNet. It replaces the most tedious task in machine learning which is gathering quality data to train the model on, by generating the needed data, this also allows agents to rapidly test new ideas and processes without needing humans to be constantly in the loop. Since this is a simulated environment, the data generated is naturally labeled, which removes the second most tedious task of manually labeling data, massively speeding up the process of model training and fine-tuning. This focus on heavily automated model training supervised by agentic agents was what allowed this project to progress so quickly and freed up much of my time to focus on the hardware aspects of this project. I go into more detail in Section 2.3. Figure 2.2 shows the MeterSim interface.

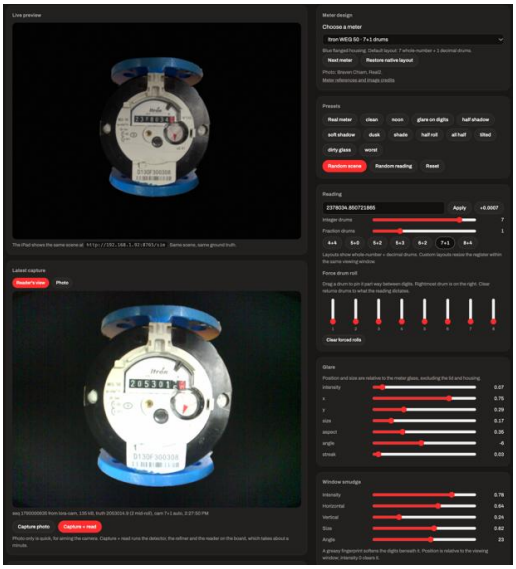


Figure 2.2 – MeterSim control interface

MeterSim features many different types of meter designs and also ways to alter the look of the meter so that it better resembles meters in the real world, it currently has 9 different meter designs that can be customized with different barrel configurations like 7 integer barrels and 1 decimal barrel (7+1), the integer barrels are typically in white or black while the decimal barrels are in red. Figure 2.3 shows the nine meter designs as rendered by MeterSim.



Figure 2.3 – The nine meter designs available in MeterSim (rendered by MeterSim)

MeterSim also allows you to simulate half rolled barrels, which allows me to train the model to predict the value of barrels that are in the middle of transitioning from one value to another. This is automatically done by the agent during training data collection (Figure 2.4).

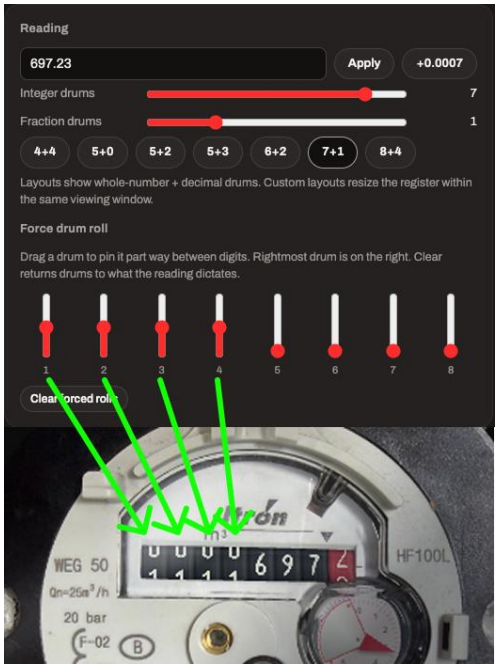


Figure 2.4 – Simulating half-rolled drums in MeterSim

MeterSim also allows you to simulate different weather and meter surface condition to better simulate how a meter left out in the elements may look like, this is included as part of the model training to make the reader more capable of reading the value in different conditions. The controls include, glare, smudge, shadow, lighting, blur, sensor noise, fog, dirt and scaling options (Figure 2.5). Common combinations are saved as presets (Figure 2.6).



Figure 2.5 – Surface and weather effects available in MeterSim



Figure 2.6 – Preset conditions available in MeterSim

MeterSim is a self-hosted web app that consists of two endpoints, the control panel "/control" and the display "/display", the control panel allows you to dial in exactly the conditions and look of the meters, while the display shows the meter to the reader. The reader is connected over WiFi on the same local network as the MeterSim instance, the reader uses this to send readings and images it has taken and receive commands from MeterSim. MeterSim is used for two purposes, model evaluation and labeled data collection. Figure 2.7 shows the display and control set-up.

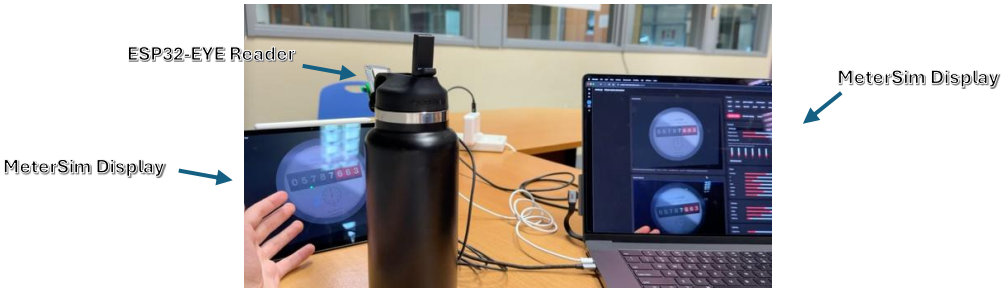


Figure 2.7 – MeterSim display and control panel with the ESP32-EYE reader

First up, model evaluation, a randomized meter and its conditions will be generated and displayed to the reader through the display, which the reader will then take an image and try to make a reading using the trained neural net. The results are then sent back to MeterSim for evaluation. Since MeterSim can generate labeled meters, the actual simulated value from MeterSim (answer) is compared against the prediction made by the reader to see if they match, it is repeated multiple times to get an approximate accuracy of the neural network model. This is performed by an agentic agent without human intervention.

Next up is collecting labeled data, which works in much the same way as model evaluation other than dropping the evaluation section and instead saving the images the reader takes to disk, along with the labeled data in JSON format from MeterSim, so that it can be used to train the model.

2.2 SimGen2

Even with all these automation workflows, there was still one major bottleneck in this system, the process of capturing images from the reader and transferring it to MeterSim takes almost 10s, when repeated over thousands of images, many hours are wasted. So my next goal was to simulate the reader's camera itself. This system was called SimGen2. It works by scaling then applying filters and warping to simulate the look of an image taken from the reader's camera. MeterSim is plugged into its input so that labeled training data can be generated end to end without ever needing any physical rig. Although the rig was kept for testing the model and also occasional data generation. SimGen2 was developed by OpenAI's ChatGPT Astra 6 entirely, the roughly 4000 images of real images taken by the reader was used to develop the filters used to simulate its look. The synthetic image is created by taking a screenshot of the MeterSim output and then passed through a warping stage that stretches the image to better fit the aspect ratio of the camera, then field blur, gain, tone curves, sensor noise and vignetting is added to the image, finally JPEG image compression of 55% (similar to quality 6 of the OV2640) is added to achieve the final look (Figure 2.8).



Figure 2.8 – The processing stages of SimGen2

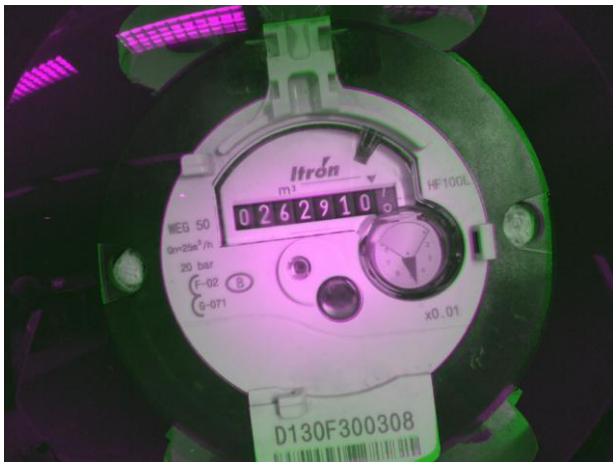


Figure 2.9 – Real capture (magenta) overlaid on its SimGen2 synthetic image (green); uncoloured areas match

SimGen2 can generate images at ~2s per image, almost 5x increase in speed compared to capturing images with the physical reader, this was used in parallel with the physical reader to generate tens of thousands of unique labeled training data which were used to create the DrumNet Simulated model. Figure 2.9 compares a real capture with its synthetic version, and Figure 2.10 shows more real and synthetic pairs.

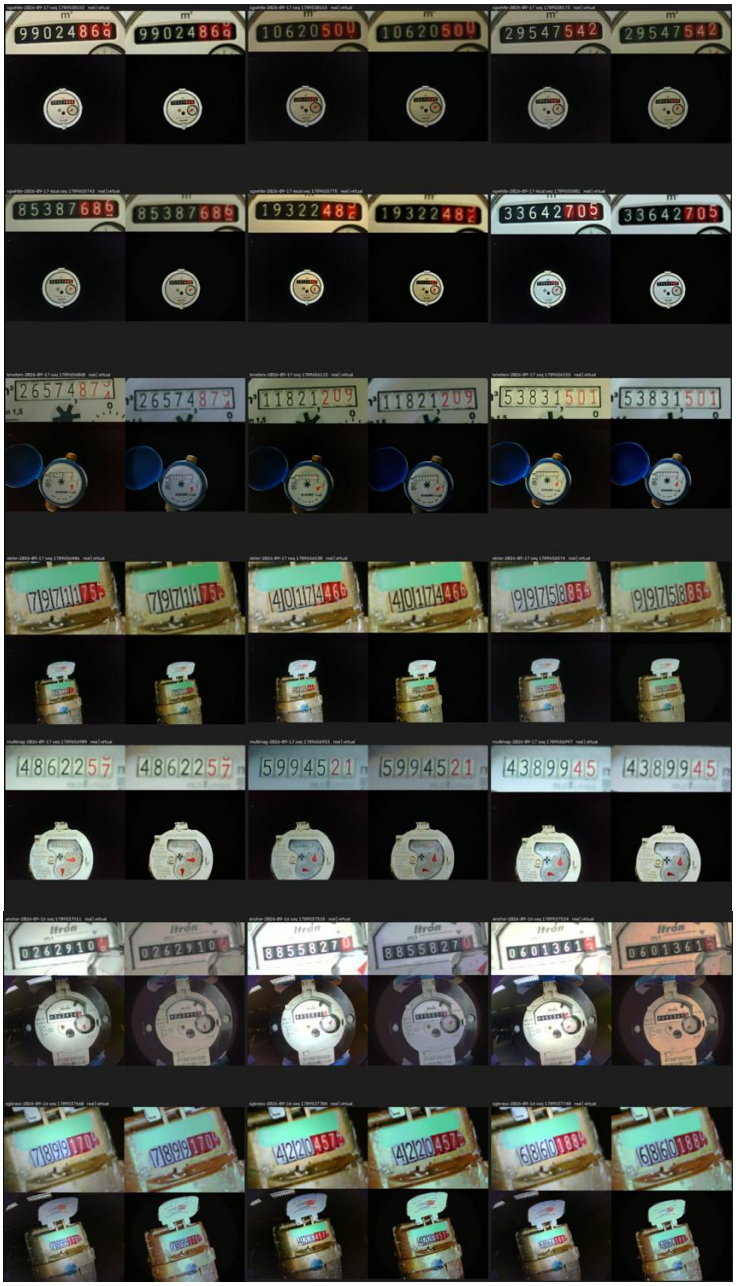


Figure 2.10 – Real and synthetic image pairs (left: real, right: SimGen2)

2.3 Agentic Workflow and Model Training

LLM based AI agents were used extensively in the development of this project. Agents were tasked to improve the full vision neural network pipeline running on the ESP32-S3 with the target of 99% Meter accuracy. The core agentic loop at play is developing a hypothesis, planning the execution, gathering relevant training data, Re-training / fine-tuning / RL the model, test for improvements or regressions in the results, then report their findings (Figure 2.11).

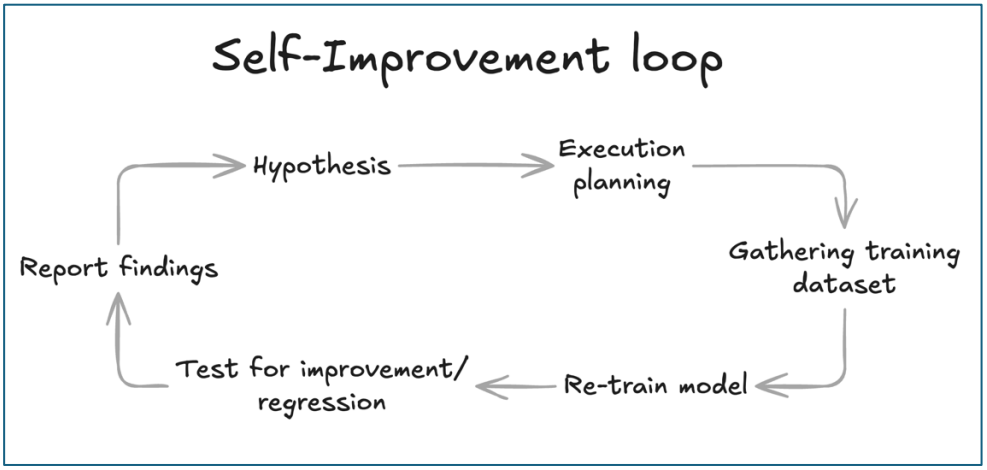


Figure 2.11 – The self-improvement loop followed by each training agent

This framework was applied to 3 agents, each working on a variant of DrumNet, the 3 different variants were Standard, Simulated and Premade:

- **DrumNet Standard** *Trained by the OCR Standard agent (Fable 5.1 & Opus 5.5)**
  Trains only from images taken by Prototype V1, starts from the base DrumNet V1 network.
- **DrumNet Simulated** *Trained by the OCR Simulated agent (Fable 5.1 & Opus 5.5)**
  Trains only from images generated by SimGen2 (Fully computer-generated images), also starts from the base DrumNet V1 network.
- **DrumNet Premade** *Trained by the OCR Premade agent (Fable 5.1 & Opus 5.5)**
  Trains from all available datasets taken by Prototype V1, generated by SimGen2 and external publicly available labeled water meter datasets which include **Badger55 (2,506 meter photos cut into 4,548 digit images; Three, 2026) and Jomjol-digits (1290 photos)**. The base model was derived from AI-on-the-edge-device's dig-cont open-source OCR model (jomjol, n.d.) Link Here.

  ** The training agents were switched from Fable 5.1 to Opus 5.5 partway through the project.*

There were 3 other agents tasked with other jobs:

- **WindowNet** *Trained by the OCR windownet agent Opus 5.5*
  Trains the spotter (DetNet) and refiner (RefNet) used to locate the drum window of a reader.
- **Sim 2 Genius** *Opus 5.5*
  The agent that is responsible for operating, maintaining and improving SimGen2.
- **OCR Coordinator** *Fable 5.1*
  The agent that coordinates and talks to all other agents to manage the project.

With 6 agents running simultaneously, they would be constantly competing for resources. As such, the scheduler was developed. The scheduler allowed the agents to book timeslots to access the shared resources available to them. 3 pieces of resource were available for the agents to book, the first two are my server (cpu compute) and laptop (gpu compute), they are collectively known as the compute resource and is used for model training. The last resource is the jetson rig, it allows the agents to flash and interact with Prototype V1 while also being able to control the display (through MeterSim) Prototype V1 is pointed at. This allows the agent to test ideas and also verify built models on real hardware (Figure 2.12 and Figure 2.13).

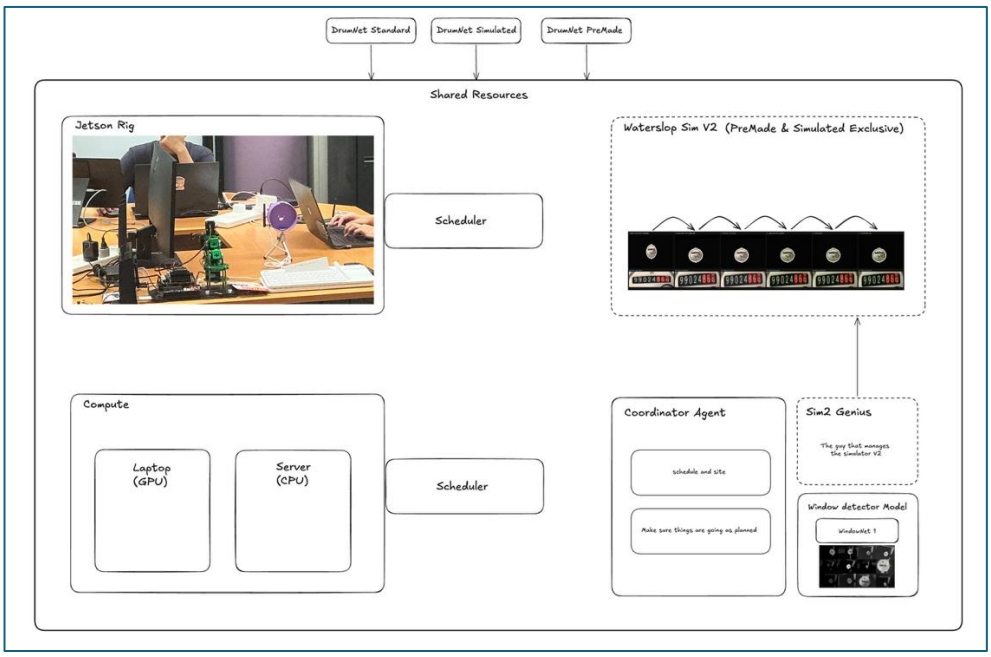


Figure 2.12 – The agents, the shared resources and the scheduler



Figure 2.13 – Scheduler bookings for the compute resources (top) and the Jetson rig (bottom)

After running for ~1.5 weeks, Standard and Premade DrumNet displayed lackluster performance, as such development for those 2 models were halted, and all resources were reallocated to DrumNet Simulated. DrumNet Simulated was now allowed to train on all available datasets, including those from the rig and the external catalogue thus, DrumNet Simulated was renamed to DrumNet. The current build of DrumNet (V10) is the highest performing NN out of the DrumNet series.

The agents ran 24/7 by implementing a "heartbeat", it's a simple script that wakes the agent up after a certain amount of time, or at a set time, to continue working. This could be scheduled when a resource becomes available or when a model has completed training. The agents were also instructed to act independently and make its own decisions, as such very little human involvement was required during this training phase, this freed up valuable time that allowed me to focus on building the hardware (Prototype V2) that DrumNet would eventually run on. This also allowed me to take my time to slowly and carefully study the models that the agent built only after completing the hardware aspect of this project, while having software and hardware aspects of the project ready in a relatively short time span.

Tailscale was at the heart of the networking aspect of this project, it allowed the agents that were running on my server at home to communicate and control the jetson rig located in school. Tailscale is a personal peer-to-peer VPN service (Tailscale, n.d.) that allows you to connect many clients across different networks under the same virtual Tailscale network, allowing for communication between the clients even if they are not on the same physical network. In my setup, the agents can communicate and use the jetson rig and the laptop(gpu) over SSH through Tailscale.

T3 Code, an open-source meta-harness wrapper for Claude Code and Codex (pingdotgg, n.d.) was used in this project to manage the agents. This allowed me to talk to and control the agents from my devices. T3 Code, being a meta-harness, allowed me to use any model from any of the two providers (Anthropic, OpenAI) and allow them to communicate with each other. T3 Code also allows me to connect multiple computers together to control them under one unified app. T3 Code worked over the Tailscale connection that was previously set up (Figure 2.14).

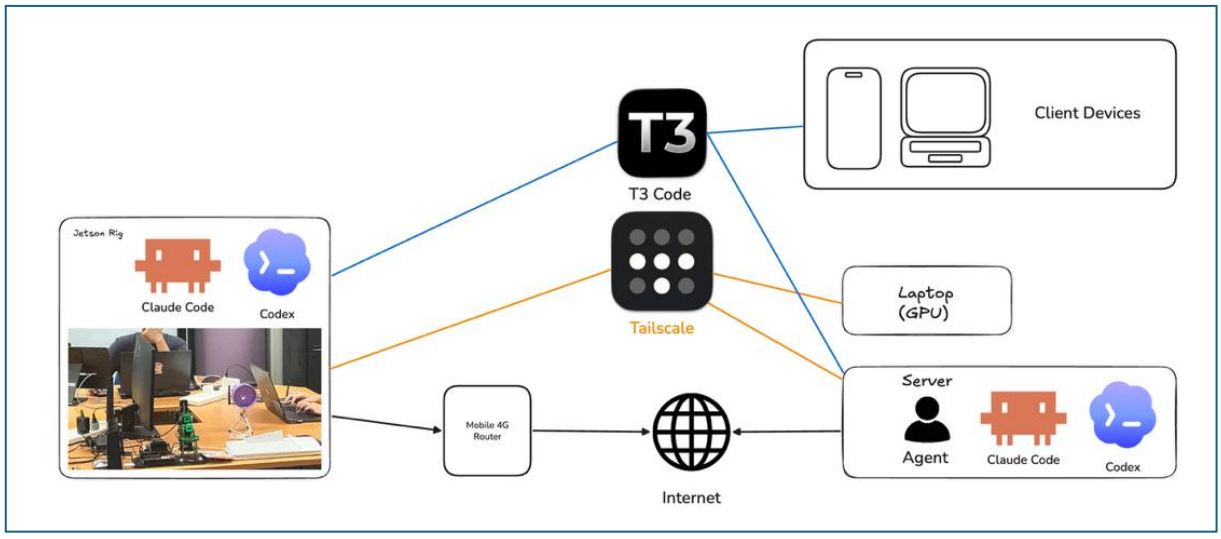


Figure 2.14 – How T3 Code and Tailscale connect the agents, machines and test rig

2.4 DrumNet Neural Network Architecture*

2.4.1 The Problem

OCR is a relatively solved problem in machine learning, with plenty of examples and available models to choose from. However, what complicated this specific application of OCR is that water meter drums have an in-between state when transitioning from one value to another, known as half-rolled drums. This makes it significantly harder to guess the true value of that drum (Figure 2.15).

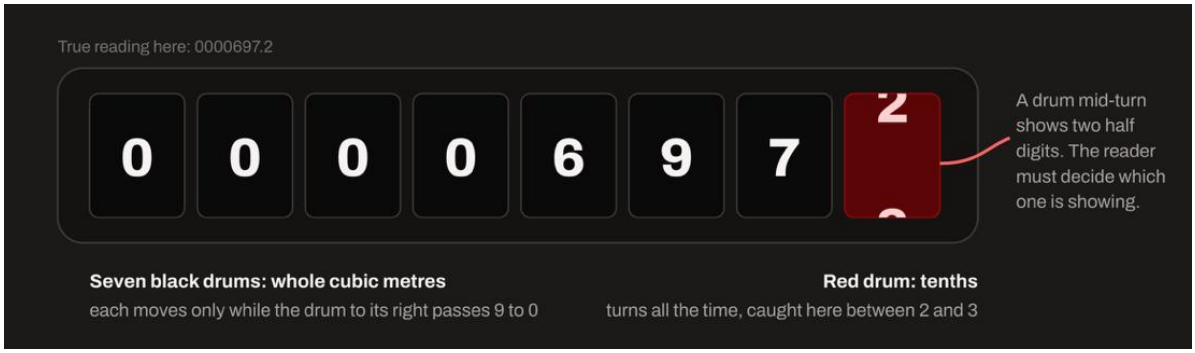


Figure 2.15 – A half-rolled drum: the last drum is caught between two digits

The second hurdle was locating the meter's digit window, then cutting out each digit correctly, and then feeding it to the OCR model. DrumNet's architecture is built heavily around solving these two problems step by step. Figure 2.16 shows an early version of WindowNet's detection and cutting results.

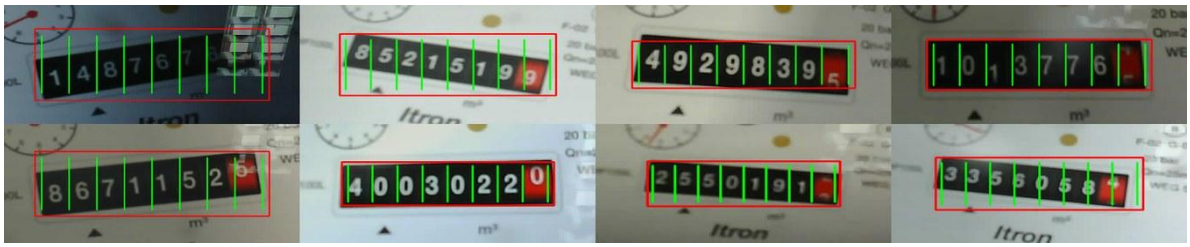


Figure 2.16 – An early version of WindowNet's detection and cutting results

2.4.2 Network Architecture

DrumNet V10 is a 693K-parameter, 3-stage CNN pipeline, the Spotter network, Edge refiner network and the reader network, the Spotter Network and the Edge refiner network is known collectively and separately as WindowNet, its job is to find the location of the drums on the meter and cut them out, the cut out images are then passed to the reader network which then makes the prediction of the final reading. DrumNet is a Mixed precision quantization network of mostly 99.95% INT8 values but retains FP16 for the most critical parts of the network that cannot survive being quantized.

It is also important to note that WindowNet does not typically run for every reading, that would be too costly. Since the meter's position stays locked, WindowNet runs during setup and stores the Region Of Interest to be reused during standard operation. If a reading turns out to be of a confidence that is lower than acceptable, DrumNet reruns with WindowNet in a sort of "recovery mode" to get a better reading.

The first step in the DrumNet V10 pipeline is to downscale the 800 x 600 px greyscale image from the camera, into a 320 x 240 thumbnail so that the spotter can run inference more quickly. Figure 2.17 gives an overview of the full pipeline.

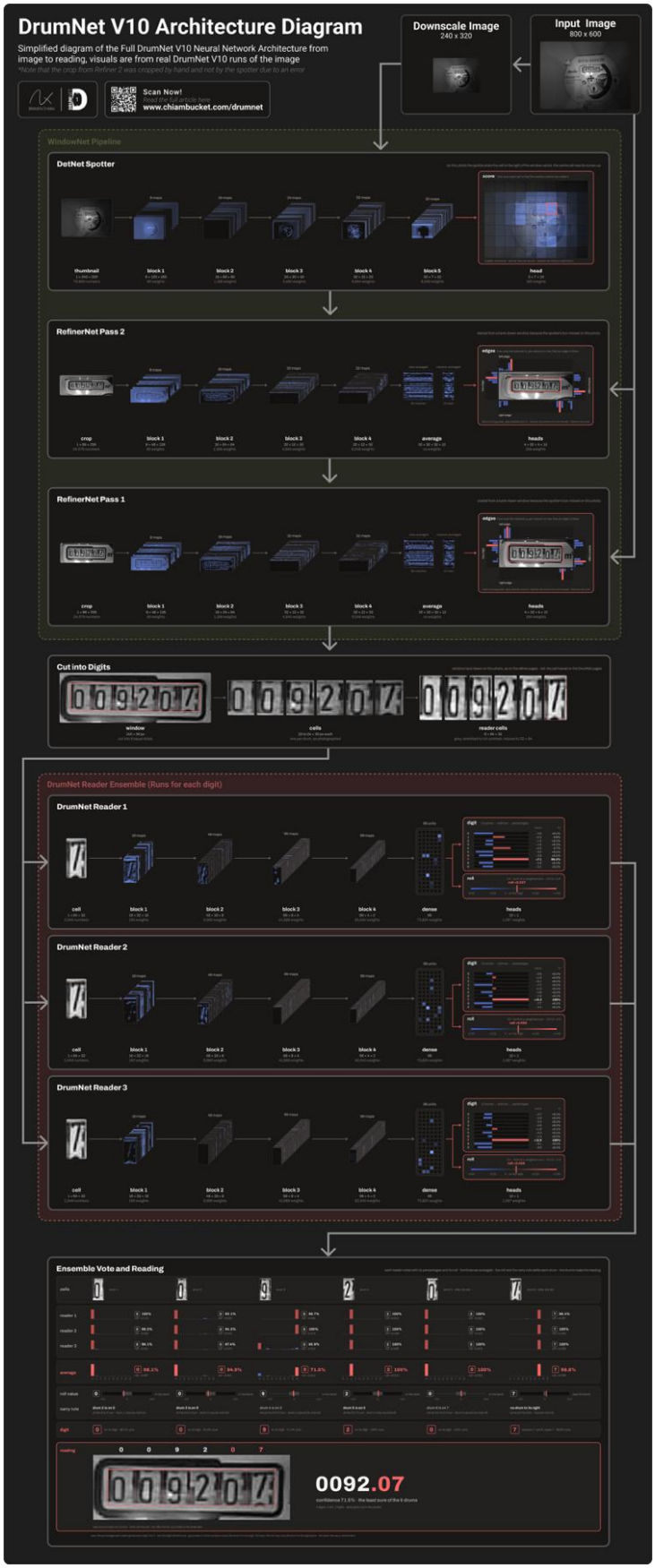


Figure 2.17 – DrumNet V10 architecture overview

2.4.3 WindowNet

The visualizations below are taken directly from an image (from prototype V2) of a meter type WindowNet has never seen before. It was therefore unable to correctly identify the ROI, however its results are promising and shows conceptually how WindowNet works

Spotter Network (DetNet): The Spotter(DetNet) is a CNN built for efficiency and rough work, I like to call it a fast search, It is made up of 5 blocks, each a 3 x 3 convolution, ReLU, and 2 x 2 Max-Pool, that shrinks the 320 x 240 thumbnail image into a 10 x 7 grid, then a 1 x 1 convolution turns the grid into x5 10 x 7 grids, each grid represents one of the values we are looking for, the score (which cell), dy (top – bottom), dx (left - right), width and height. The score map is used to find the cell (highest value) on the map that best corresponds to the center of the meter's digit window, the position of that cell is mirrored to the other layers to get the position and size of the Region Of Interest (ROI). The ROI is scaled and used to crop the original image, since DetNet's work is rough, the crop is typically x1.5 wider and x2.4 taller than the ROI. This allows the refiners to work with a more focused image instead of searching for the window through the whole image. If confidence is low DetNet retries with an alternative DetNet model (Figure 2.18).

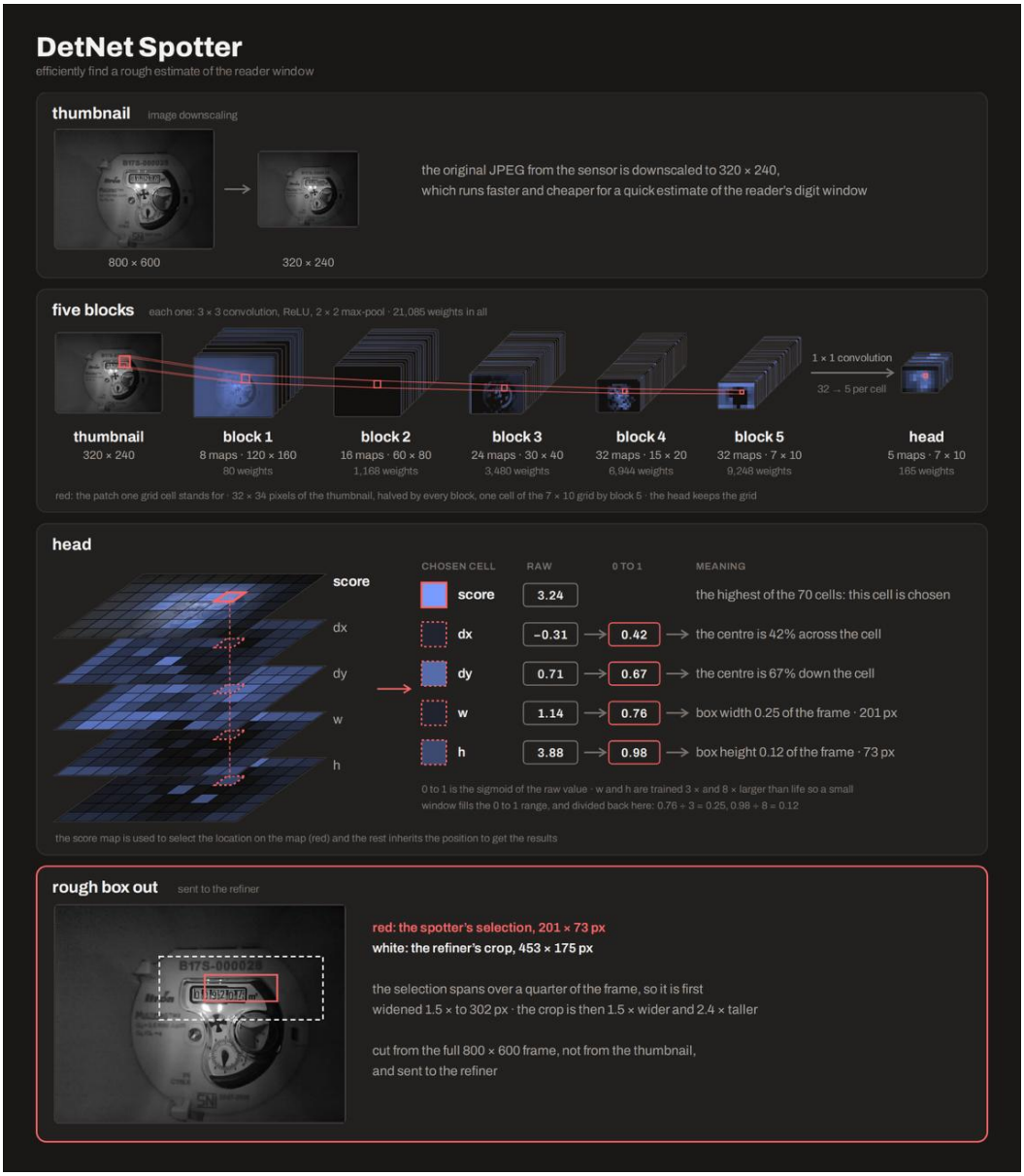


Figure 2.18 – The Spotter network (DetNet)

Refiner Network (RefNet): The Refiner network refines the selection of the ROI, it runs twice, first from the output of the spotter, its generated new ROI is expanded (x1.5 and x2.4) and passed back into the same refiner for a second refinement run. The architecture of RefNet is like DetNet, however it differs in its heads. RefNet is much more detailed and computationally intensive as compared to DetNet. RefNet is 4 blocks of 3 x 3 convolution, ReLU, and 2 x 2 Max-Pool operations that turns the 256 x 96 cropped image into 32 maps of 32 x 12 values. Each column of each map in the 32 x 32 x 12 map block is compressed into an average of each column which produces a 32 x 32 column average map, likewise each row of the 32 x 32 x 12 map block is compressed into an average of each row which produces a 32 x 12 row average map.

The column average map is run through a 1 x 1 convolution block which produces a left score and offset, and a right score and offset, the highest score is used to locate the position on the offset slice, that offset value is used to calculate where that edge is on the map, the same is done for the row average map to get the top and bottom edge of the reader window (Figure 2.19).

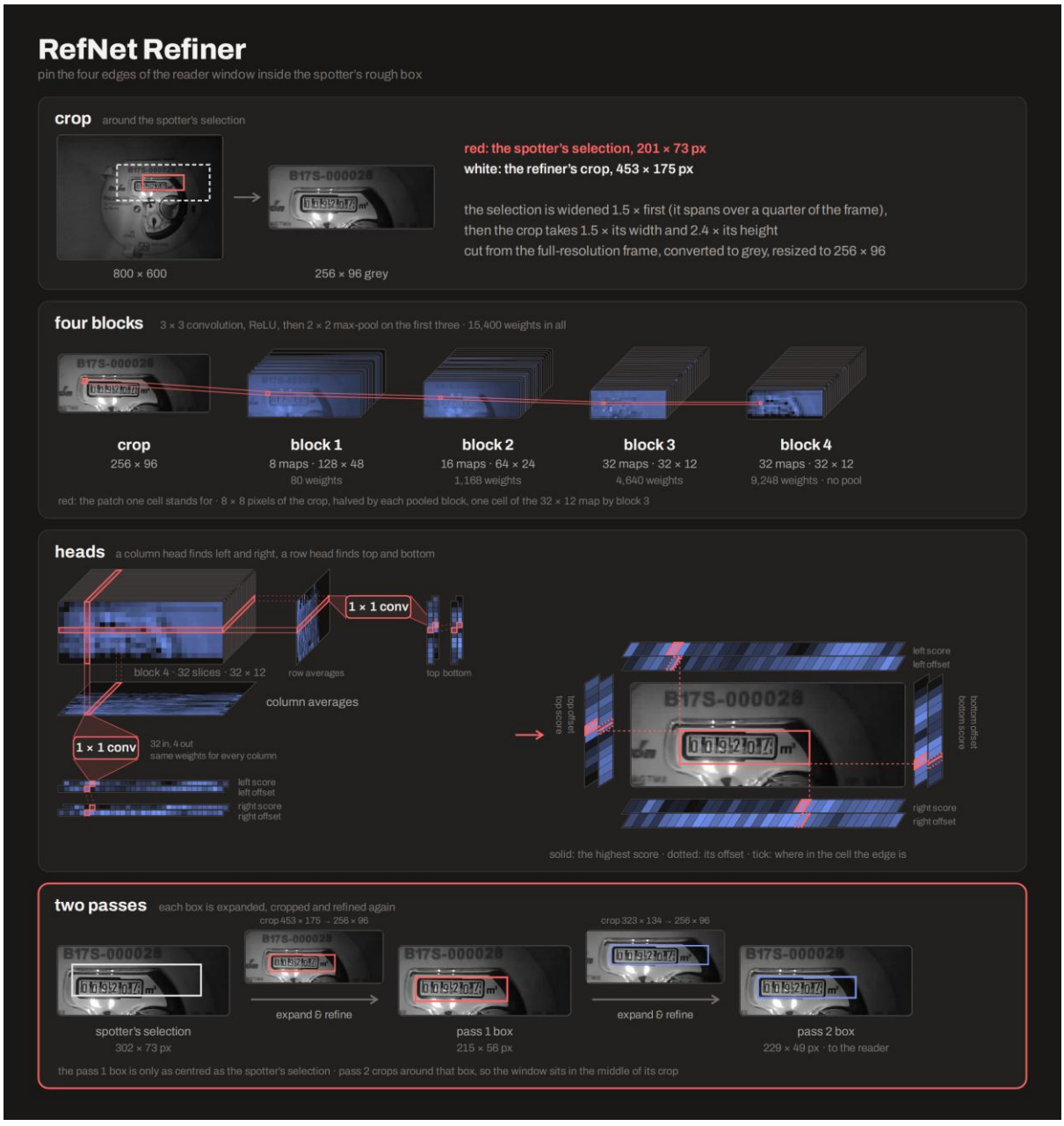


Figure 2.19 – The Refiner network (RefNet)

Cutting out the digits: the drum layout, for example 4 whole drums and 2 decimal drums (4 +2), should have been set during setup. The layout is used to evenly slice the ROI into slices of each drum (cell), then each cell will be passed to DrumNet to read the digit on the drum.

Note that since WindowNet could not correctly identify the ROI, the ROI used in Figure 2.20 (and in later figures) was drawn by hand, not from the output of WindowNet.



Figure 2.20 – Cutting the region of interest into individual drums (ROI drawn by hand)

2.4.4 DrumNet Reader Ensemble

The DrumNet reader ensemble is a collection of 3 readers networks, all 3 networks share the same architecture, however, they are each trained from different random starting seeds, all 3 perform the same purpose of finding the digit and roll value of a given cell. Each cell is fed into all 3 readers which produces 3 slightly different results. All 3 results are then averaged in the voting phase to get the final reading.

Each reader (206,619 parameters) is a 4-block 3 x 3 convolution, ReLU and Max-Pool operation, that converts the 32 x 64 cell image into 96 feature maps of 2 x 4 (block 4). These 96 feature maps are then flattened by rearranging each 2 x 4 feature map into a 1 x 8 list; it is then combined to form a long list of 768 values.

In the dense layer, the weighted sum of the 768 numbers and each of the 96 weights of the dense layer are passed through a ReLU activation function to get 96 values/units, these 96 units will be used to compute the reading and roll value.

To predict the digit in the cell, the weighted sum of each possible class (from 0 to 9) is taken with the 96 units to get a probability distribution of the classes. A SOFTMAX function is applied to normalize the probability distribution and the class with the highest probability will be the selected answer. The confidence value will therefore be the probability value of the selected class.

To predict the roll value, the weighted sum of the 96 units and the roll weights is taken and then squished with a tanh function to get a roll value between +0.5 and -0.5. The value is used to estimate how far the drum has rolled and in which direction (Figure 2.21).



Figure 2.21 – A single DrumNet reader network

Ensemble Vote and Reading: In this stage, the 3 sets of probability values (after SOFTMAX) from the 3 readers are averaged together, each reader has the same "voting power" as such the third reader was added in as a tie breaker. The resulting answer would be the class that most of the readers accept, the new confidence and roll is averaged to give the reader's confidence and roll value.

Finally, the carry rule is applied to settle each drum to its final value, this fixes the edge cases between the transition of two drums simultaneously. The carry rule works by checking the value of the drum to the right of it, if the value to the right of the drum is 0 or 9, it indicates that the drum on the left could be in a state of transition and the roll value is acceptable, but if a digit to the right is between 1 and 8, the digit on the left cannot be in a state of transition, as such the roll value is negated and the digit is absolute. This allows the carry stage to catch edge cases from 2 drums transitioning together, for example, if a value of two drums that total 79 are transitioning to 80 together, the reader may have a chance of reading as 89, using the carry rule, the reader knows to pick the preceding number instead (Figure 2.22).

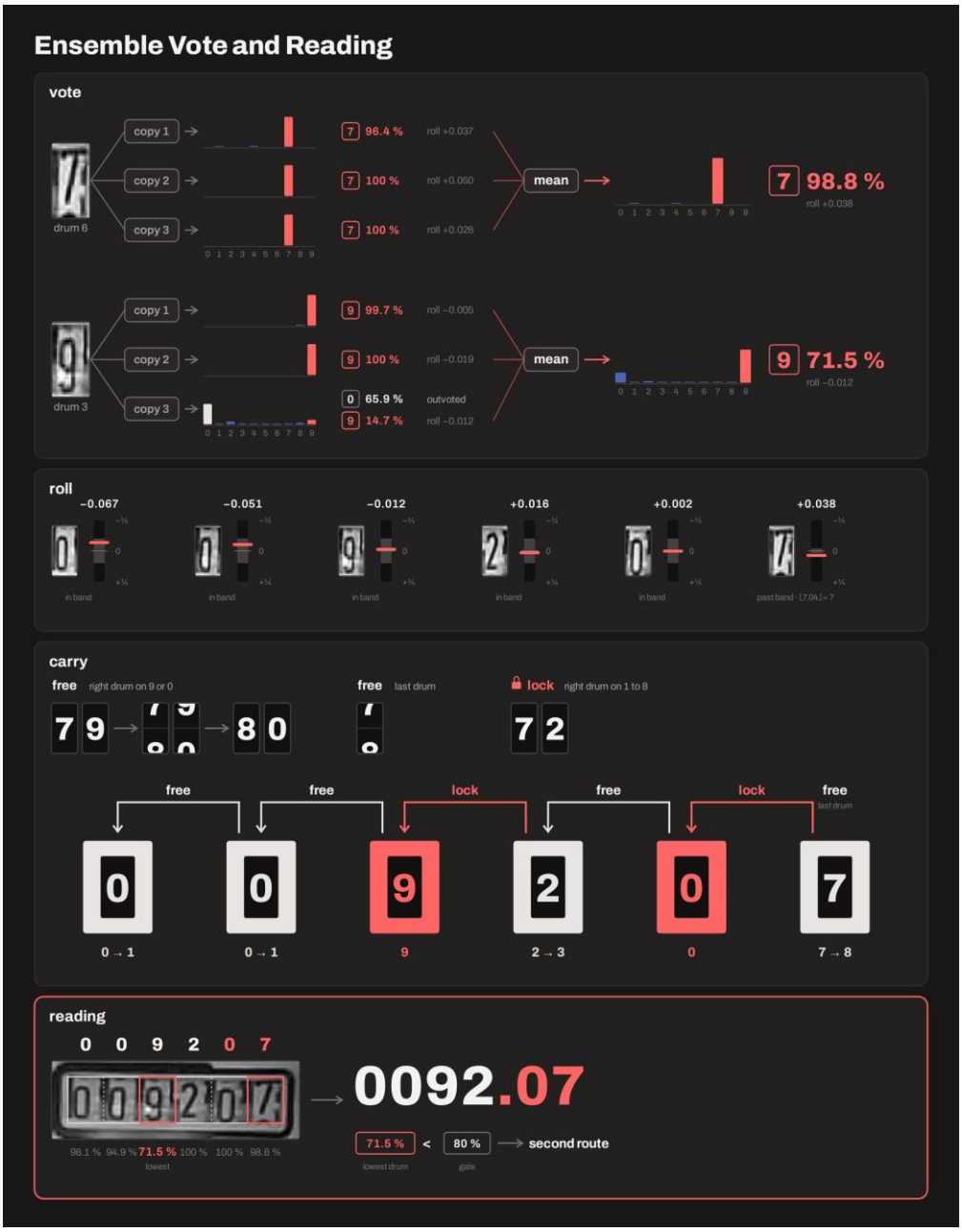


Figure 2.22 – Ensemble vote, carry rule and final reading

2.4.5 DrumNet V10 Summary

DrumNet originated from DrumNet Simulated and was renamed DrumNet. **Error! Reference source not found.** summarises the key figures of DrumNet V10.



Item	DrumNet V10
Parameters	**693K** (DrumNet ~620K + WindowNet ~73K)
Size on board	**~920 KB** (DrumNet 636 KB int8 + WindowNet ~285 KB)
Read time	**~2.1s** (with WindowNet)
Training data	**DrumNet Reader: 197,204 Images** WindowNet: 12,000 simulator photos + ~5,300 real-meter photos
Training time (DrumNet only)	**~6** h on an RTX 3080 laptop (fine-tuned from V9b); WindowNet not counted
Accuracy: full pipeline	**56% (808/1,440)**
Accuracy: training fit (DrumNet only)	99.8%
Accuracy: pooled test set (DrumNet only)	93.2% (538/577)
Accuracy: on board (DrumNet only)	85.8%

Table 2.1 – Key figures for the on-board

** Please be advised that this section scores a Level 4 in the scale of AI use, separately from the rest of the project.*



2.5 Firmware (Agentic Workflow II)

An Astra 6 agent was tasked with writing the firmware for the reader (Prototype V2) and the M5Stack Cardputer Adv (ConfigKing; M5Stack, n.d.). The Cardputer's role in the project is to be the configuration console that allows onsite engineers to configure the readers for deployment, it also allows configured readers to be reconfigured. The agent was also told to only write code using Fable 5.1.

My Laptop along with the LoRaWAN gateway (graciously provided by Mr Lee Chang Fatt) and the jetson nano were setup on the same network, The cardputer and Prototype V2 were connected over USB serial to my laptop. All 4 devices were made available to the agents. The setup process will be covered in Section 3.6.9. Figure 2.23 shows the setup.

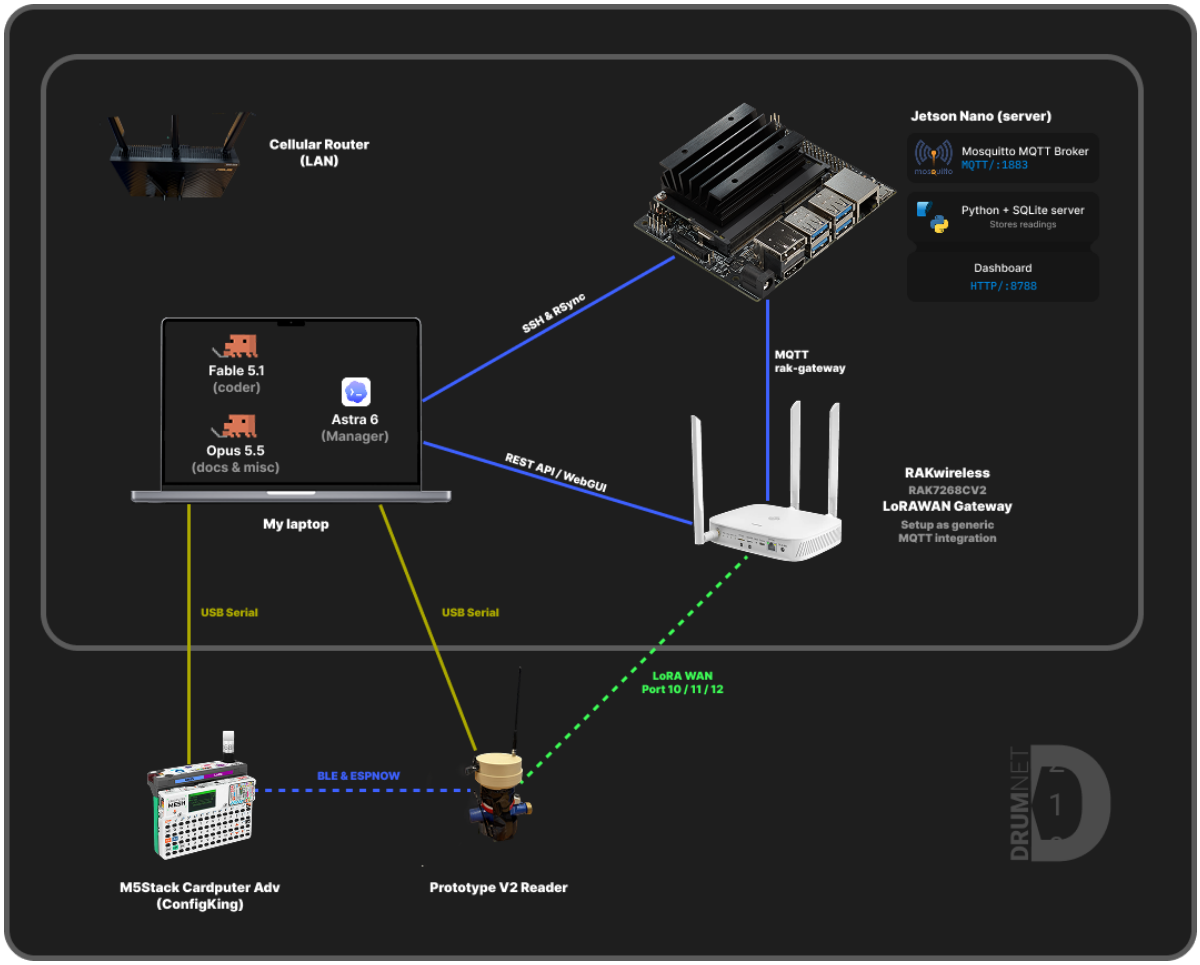


Figure 2.23 – Firmware development setup made available to the agents

2.5.1 Reader Firmware

On a standard deployed reader, the reader wakes up from sleep, restores the reader configuration and LoRaWAN session from Non-Volatile Storage (NVS). The camera is next initialized and a capture is taken with the LED lights. Next the image is run directly through the DrumNet readers using the stored ROI from setup (no WindowNet), only if the DrumNet readers report a low confidence, will the full DrumNet Pipeline run as a backup. If a failed first attempt is detected, it will be stored in the NVS for future evaluation, a low-confidence warning will also be reported in the dashboard. Once the reading is ready, the LoRa link is switched on and reading will be sent to the LoRa gateway. Finally the reader shuts off all peripherals and goes into deep sleep to wake up at the next RTC triggered cycle.

Since the reader is in deep sleep most of the time, pressing the boot button wakes it for configuration. Figure 2.24 shows the firmware flow.

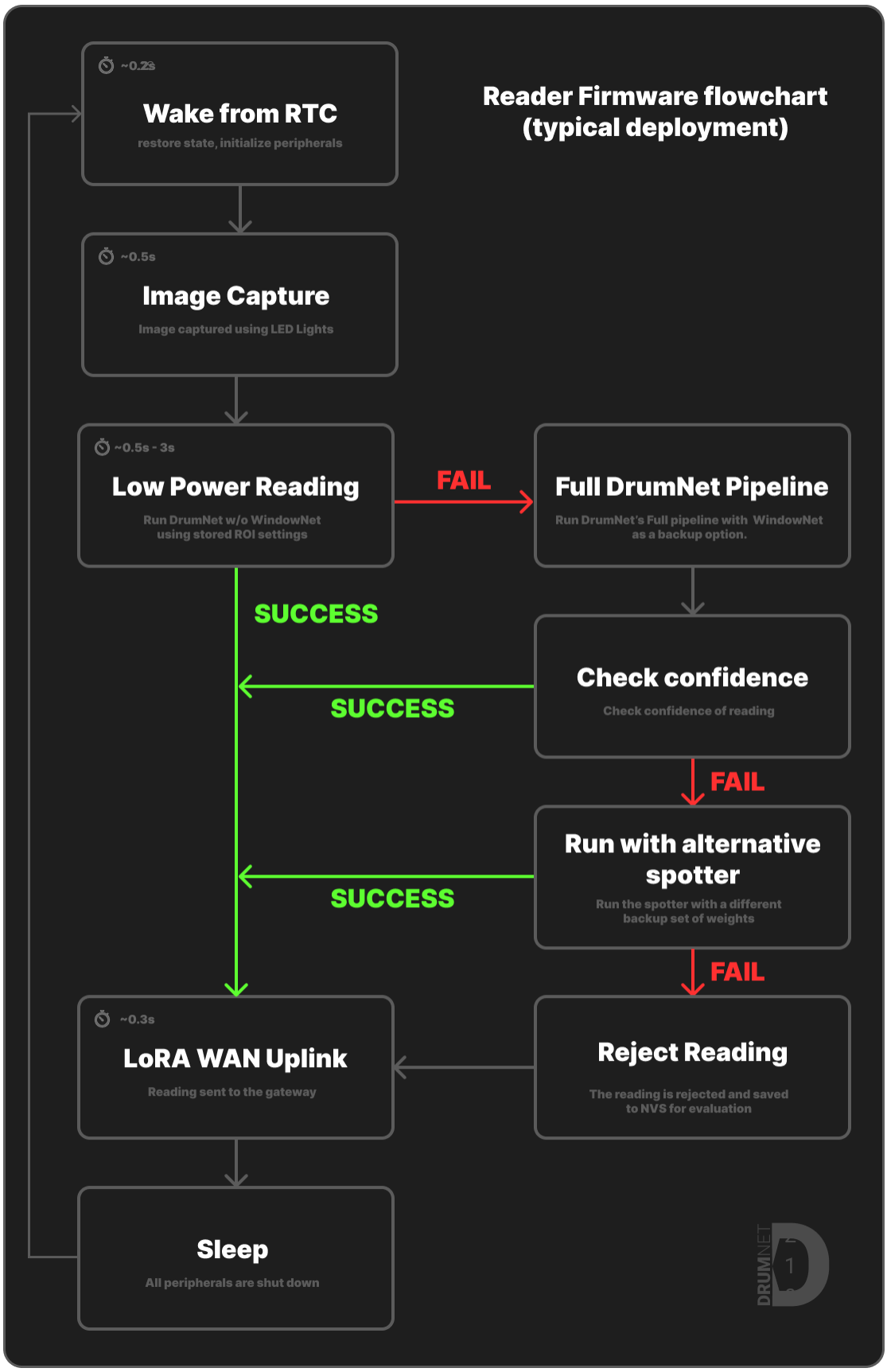


Figure 2.24 – Reader firmware flowchart for a typical deployment

2.5.2 Dashboard

After the reading is made, it is sent to the LoRaWAN gateway. The LoRaWAN forwards the reading over MQTT where it is stored on the jetson nano. The jetson nano serves the Dashboard as a single HTML file, the JavaScript on the page updates the values through the data stored on the jetson nano. The dashboard is a demo and is not designed to scale to real deployments (Figure 2.25).

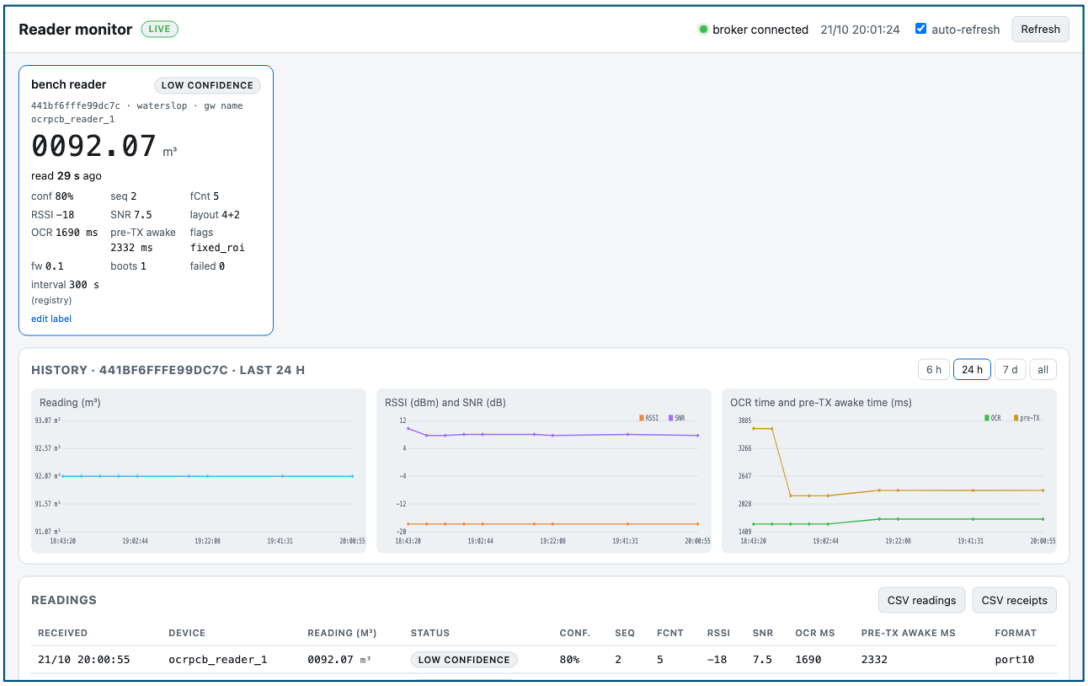


Figure 2.25 – Reader monitor dashboard served by the Jetson Nano

2.5.3 LoRaWAN and BLE Payloads

Figure 2.26, Figure 2.27 and Figure 2.28 show the LoRaWAN uplink payloads, and Figure 2.29 shows the BLE protocol used between ConfigKing and the reader.

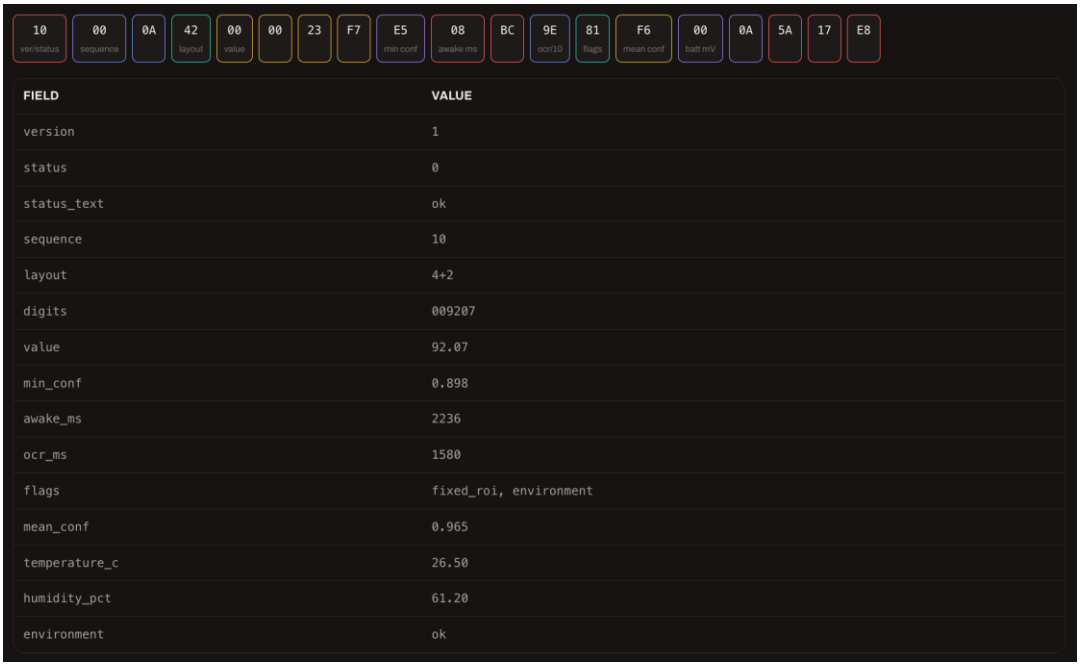
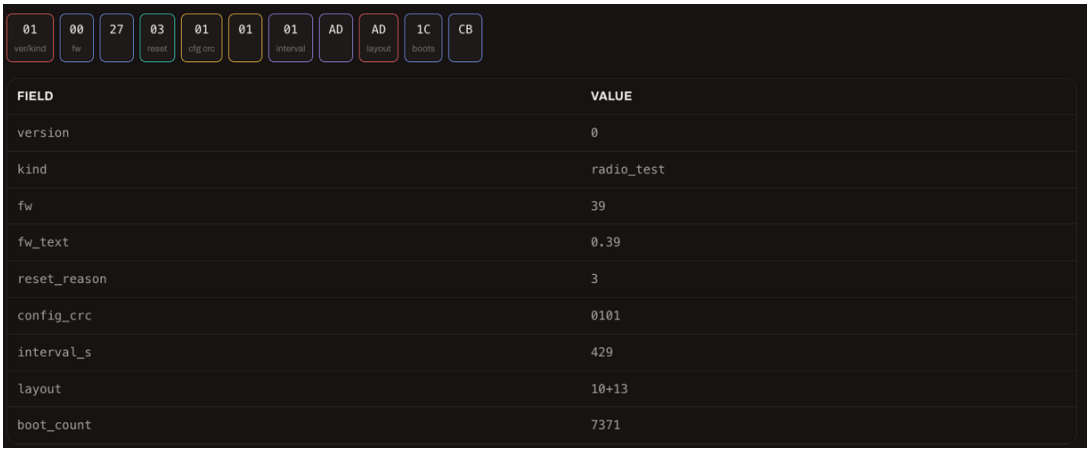
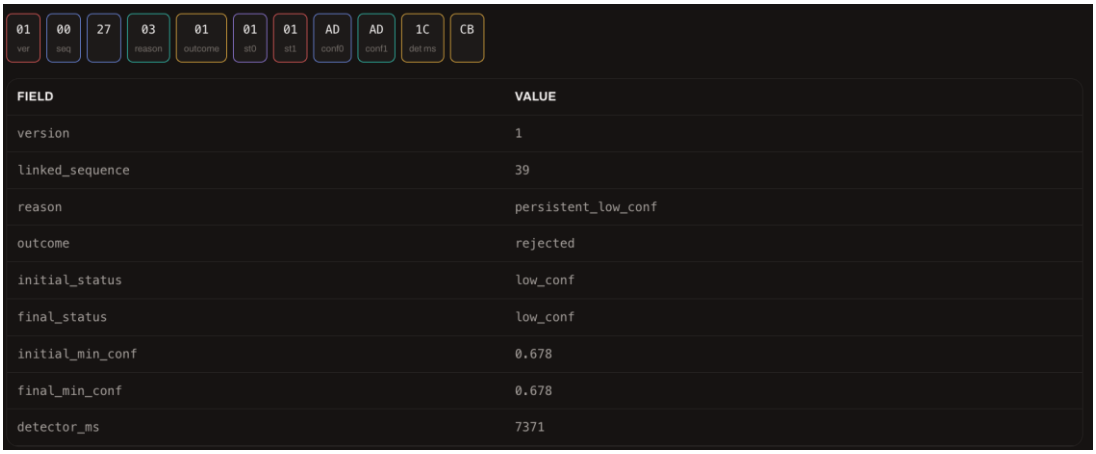


Figure 2.26 – Reading payload (LoRaWAN port 10)



FIELD	VALUE
version	0
kind	radio_test
fw	39
fw_text	0.39
reset_reason	3
config_crc	0101
interval_s	429
layout	10+13
boot_count	7371

Figure 2.27 – Radio test payload (LoRaWAN port 11)



FIELD	VALUE
version	1
linked_sequence	39
reason	persistent_low_conf
outcome	rejected
initial_status	low_conf
final_status	low_conf
initial_min_conf	0.678
final_min_conf	0.678
detector_ms	7371

Figure 2.28 – DrumNet recovery event payload (LoRaWAN port 12)

BLE ITEM	VALUE
Name	WSR-DC7C (WSR- plus the last 4 hex of the DevEUI)
Service	6e400001-b5a3-f393-e0a9-e50e24dcca9e (Nordic UART layout, custom semantics)
RX/TX/IMG	...0002 write, ...0003 notify (reply lines), ...0004 notify (2-byte LE chunk index + up to MTU-5 bytes of JPEG)
Advertisement	manufacturer 0xFFFF, 'W', state byte (0 discovery, 1 configured setup), 8-byte DevEUI
Security	LE Secure Connections, Just Works, bonding, encrypted characteristics, one connection, 3 bonds max. The physical commissioning window is the authentication.

COMMAND	EFFECT
HELLO	name, fw, deveui, state, configured, key present, uptime. First thing the wizard sends.
CFG GET / SET <json> / COMMIT / DISCARD / RESET	Staged partial JSON, validated as a whole; nothing persists until COMMIT. Identity changes clear the session so the next wake rejoins.
KEY SET <32 hex>	Stages the AppKey; the reply never echoes it and no command reads it back (CFG GET reports only app_key_set). A setup.allow_key_show flag exists in the config model, default false, but no command in commands.cpp or main.cpp honours it.
PREVIEW [maxw]	One lit frame, software-shrunk JPEG (default 240 px wide) streamed in chunks; IMG MISS re-requests chunks.
ROI SET x y w h / ROI FIND / ROI AUTO	Fixed box in SVGA pixels (validated against the layout); FIND and AUTO are experimental detector paths.
READ / ANNOT	Full capture + OCR with the staged config; status, digits, confidences, box, timings, then the annotated JPEG.
FRAME [maxw] [x y w h] / FRAME RAW	Re-sends the frame of the last capture without a new capture or LED: FRAME RAW is the camera's own 800x600 JPEG (about 22 kB), FRAME 480 a 400x300 re-encode, a rectangle keeps full resolution. Headers carry src= , scale= and frame=n ; READ replies carry the same frame=n , so a client never pairs an old reading with a newer picture. ANNOT refuses with ERR ANNOT frame replaced by PREVIEW, run READ . Reader 043149d9.
OK END ... retries= drops=	BLE image notifications are sent through the host's own notify call so a refusal for lack of buffers is seen: the reader waits 8 ms and retries, up to 40 times per chunk, and reports the counts. Three repeated FRAME RAW of one frame: 22102 B, 93 of 93 chunks, 1.423 to 1.429 s, drops 0 (reader-ble/repeat.log).
RADIO TEST	Joins if needed, sends one confirmed port 11 frame, reports ack, RSSI, SNR, DR, bytes, trimmed, ms. Real result: joined=new fcnt=0 ack=1 rssi=-16 snr=10.8 dr=3 bytes=14 trimmed=0 ms=6992 .
STATUS / DONE / SLEEP / FACTORY RESET	Counters JSON; finish setup (port 11 status, then sleep); immediate sleep; wipe the reader namespace and reboot into discovery.

Figure 2.29 – BLE protocol between ConfigKing and the reader

3. Hardware

3.1 Processor (MCU)

After screening and evaluating multiple processors, The ESP32-S3 with 8MB of PSRAM came out to be one of the more balanced options. I chose it primarily for its ease of access and my general comfort working with Espressif 's chips. While searching for suitable chips for this project, I listed down some key considerations and specs that were a must-have:

1. **Digital Video Port (DVP) support (non-negotiable)**
   DVP is used by most low cost cameras for microprocessors specifically the OV2640 camera that I will be using for this meter.
2. **Hardware Accelerated vector / matrix instruction sets (non-negotiable)**
   Vital for running neural networks quickly and efficiently on microprocessors that already have a limited energy budget.
3. **Low cost**
   Needs to be relatively low cost.
4. **Low power**
   The keywords are high efficiency, low power and sufficient performance, while being able to support the 2 yearlong lifespan of this reader.

There are 3 chips from Espressif that fit the non-negotiable criteria. The P4, S31 and the S3. The ESP32-P4 is the most powerful of the 3, It is primary targeted at multimedia applications where displays and Human Machine Interaction (HMI) heavy compute is needed, much too overkill for this project. The S31 looks to be the second best contender with plenty of welcomed security upgrades like glitch detection and APM that help protect against physical tampering, it also bumps the available GPIOs to 60 which would alleviate my current GPIO constraints and its newer and faster. But the S31's deep sleep current draw is 12uA as compared to the S3's 7uA, almost double the current drawn (Espressif Systems, 2026c, 2026e). The S3 is still the most reliable and mature MCU platform having been released and battle-tested for nearly 6 years (Espressif Systems, 2020) with plenty of community support and documentation, while the P4's current chip revision (v3.x) was only released in May 2026 (Espressif Systems, 2026a) and the S31 only entered mass production in July 2026 (Espressif Systems, 2026d). That was why I chose to go with the ESP32-S3 for this project with 8MB of PSRAM.

Even though I chose the ESP32-S3, it comes with some Espressif specific quirks for example, Espressif does not match the reliability and security reputation that industry giants like STMicroelectronics, Nordic, and NXP have. The next best non-espressif option would be from the Alif Ensemble E3 series (Alif Semiconductor, n.d.), but this is 5.5 to 8.1 times more expensive than the ESP32-S3, depending on order quantity (Table 3.1). The ESP32-S3 is purpose built for IoT edge-AI applications like this, which is why I plan to continue using it for this project.

Table 3.1 – Unit price of the ESP32-S3R8 vs the Alif Semiconductor Ensemble E3 by order quantity (USD)

Quantity	ESP32-S3R8 (LCSC)	Alif E3 (Mouser)	E3 ÷ S3
1	$3.02	$16.72 (AE302F80F55D5AE-Y)	5.5×
100	$2.19	$15.82 (AE302F80F55D5AE-Y)	7.2×
~1,000	$2.06 (1,000+)	$16.78 (AE302F40C1537LE-T, 1,500 reel)	8.1×

Sources: LCSC – ESP32-S3R8 (C2913194); Mouser – Alif Semiconductor E3 series; Findchips – ESP32-S3R8.

Table 3.2 – Comparison of the Espressif ESP32-S31, ESP32-S3 and ESP32-P4

Feature	ESP32-S31	ESP32-S3	ESP32-P4
Core & Memory			
CPU Architecture	Dual-core 32-bit RISC-V®	**Dual-core 32-bit Xtensa® LX7**	Dual-core 32-bit RISC-V®
Max Frequency	320 MHz	240 MHz	**400 MHz**
AI Instruction Accel	**Accelerator Instruction Set Supported**	**PIE Instruction Extension**	**Supports custom AI and DSP extensions (XespV)**
Peripherals			
GPIO	**60**	45	55
LCD/Camera	RGB · I80 · DVP	RGB · I80 · DVP	RGB · I80 · DVP
JPEG Codec	**Supported**	Not Supported	**Supported**
Security	**Secure Boot, Crypto Accelerators (AES, ECC, HMAC, RSA, SHA, RSA_DS, ECDSA_DS), SDC, XTS_AES, TRNG, Key Manager, Glitch Detector, DPA, APM, TEE**	**Secure Boot, Flash Encryption, Crypto Accelerators (SHA, AES, RSA, HMAC, RSA_DS, RNG)**	**Secure Boot, OTP, Cryptographic and Security Components (AES, ECC, HMAC, RSA, SHA, RSA_DS, ECDSA_DS, XTS_AES, TRNG), Key Manager, PMS**
Power Management			
Light-sleep Current	**50 uA**	240 uA	75 uA
Deep-sleep Current	12 uA	**7 uA**	12 uA
Low Power Coprocessor	Not Supported	**Supported**	**Supported**

Sources: Espressif Systems (2026b, 2026c, 2026e).

3.2 Battery Technology

The two most important factors when picking the right battery technology and chemistry for this project is the ability for it to adapt to a wide range of temperatures and weather conditions while also having a low self-discharge rate. The readers could be placed in a cold, dark and damp utility closets, or placed in extreme heat under direct sunlight, and still be required to operate flawlessly. The batteries would also need to have a very low self-discharge rates as the reader would be in deep sleep for most of the time during its ~2 year long life cycle. The first choice for most ultra long lifecycle IoT applications would be to use Lithium coin cell batteries, as they have extremely low self-discharge rates that allow them to operate over many years without maintenance. But it does not have nearly enough energy capacity to run the OCR model, operate LoRa and power the camera while lasting ~2 years. That is why I decided to go with Li-$SOCl_2$ batteries (bobbin/spiral not decided), they are already widely used in industrial IoT applications where conditions could vary drastically. They also have a relatively low ~1-3% / Yr self-discharge rate.

Table 3.3 – Comparison of primary battery chemistries used in IoT devices

Chemistry	Example cell	Nominal V	Energy density (Wh/kg · Wh/L)	Self-discharge	Pulse current	Operating temp	Typical IoT use	Watch out for	Approx. price (USD/cell)
Li-SOCl2 bobbin	Saft LS33600, Tadiran TL-5930, EVE ER34615	3.6	~610 · ~1,180	<1%/yr	Low (D cell: 250 mA continuous, 400 mA for 0.1 s)	−60 to 85 °C	Water/gas meters, asset trackers, 10–20 yr sensors	Passivation and voltage delay after idle periods, poor at high current, can't be charged	~$8–12 (EVE); ~$19–30 (Saft/Tadiran D)
Li-SOCl2 spiral	Saft LSH20	3.6	~490 · ~930	~3%/yr	High (1.8 A continuous, 4 A pulse)	−60 to 85 °C	Devices with motors, valves, GSM/NB-IoT bursts	Higher self-discharge, lower capacity than bobbin	~$28–42 (D)
Li-SOCl2 + hybrid layer capacitor	Tadiran PulsesPlus	3.6	Same as bobbin	<1%/yr	Very high (HLC: 2 A pulses)	−40 to 85 °C	Cellular/LoRa meters needing long life plus big TX pulses	Cost, larger pack, fewer suppliers	~$20–40 (AA/C); ~$40–60 (D)
Li-MnO2 cylindrical	CR123A, Saft LM33600	3.0	~270 · ~600 (CR123A)	~1–3%/yr	High (CR123A: 1.5 A continuous)	−40 to 70 °C	Smoke alarms, smart locks, security sensors	Lower capacity than Li-SOCl2	~$2–6 (CR123A); ~$30+ (LM33600)
Li-MnO2 coin	CR2032, CR2450	3.0	~225 (CR2032)	~1%/yr	Very low (tens of mA; needs a bulk cap for BLE)	−30 to 70 °C	BLE beacons, tags, RTC backup	High internal resistance, capacity falls sharply under pulses	~$0.30–1.50
Li-FeS2	Energizer L91 (AA)	1.5	~300–330 · ~550–600	~1%/yr (20-yr shelf rating)	High (2.5 A continuous)	−40 to 60 °C	Drop-in AA replacement, field-swappable sensors	1.8 V fresh, can over-volt parts designed for alkaline	~$1.50–3
Alkaline	Energizer E91 (AA)	1.5	~150–170 · ~450 (low drain)	~2–3%/yr	Moderate, capacity collapses at high drain	−18 to 55 °C	Consumer gadgets, remotes, low-cost sensors	Leakage, poor in heat and cold, sloping voltage	~$0.30–1
Zinc-air	Hearing-aid cells, some trackers	1.4	~400+	Low while sealed, dries out within weeks once opened	Low	~0 to 50 °C	Short-life wearables, hearing aids	Needs airflow, humidity-sensitive, short life once activated	~$0.30–0.80
Silver oxide	SR626, SR44	1.55	~130	~5–10%/yr	Low	−10 to 60 °C	Watches, small medical sensors	Tiny capacity, expensive	~$0.50–3

Prices are approximate single-cell retail prices from US battery distributors (October 2026); volume pricing is typically 10–35% lower. Energy density and current figures are typical datasheet values (e.g. Saft, 2026).

3.3 Power Optimization

Power optimization is a key aspect for this project as it needs to run for very long durations (up to 2 years) on a set of batteries. With that in mind, the ESP32-S3 is relatively efficient at heavy tasks like vector math that is required for neural networks. But it is nowhere near as efficient as Nordic's nRF lineup. As such this section talks about the strategies I will attempt to implement, to reduce energy usage and increase efficiency of the ESP32-S3 Chip. And thus, increasing the operating lifespan of the device. Note that the wireless radios of the ESP32-S3 will be turned off during typical operating conditions in deployment, they will only be used for initial configuration.

3.3.1 Software-Based Optimizations

Although this is still work in progress, this will be one of my focuses on extending the runtime and life cycle of the OCR meter reader. Most of them have already been implemented but their energy impact has not been conclusively measured and requires more testing and evaluation.

1. **In Camera Sub Sampling**

The OV2640 supports up to a resolution of 1600x1200, but also supports sub-sampling modes of 800x600 SVGA and 400x296 CIF (OmniVision Technologies, 2006). Based on my testing (Figure 3.1 and Figure 3.2), the 800x600 SVGA mode yielded the best balance of quality and speed. In this case, speed is important as the less time the camera stays on, the less energy it consumes. A lower resolution of 800x600 also means the neural net needs to process 4x lesser information as compared to 1600x1200, this will result in faster processing and lesser energy use.

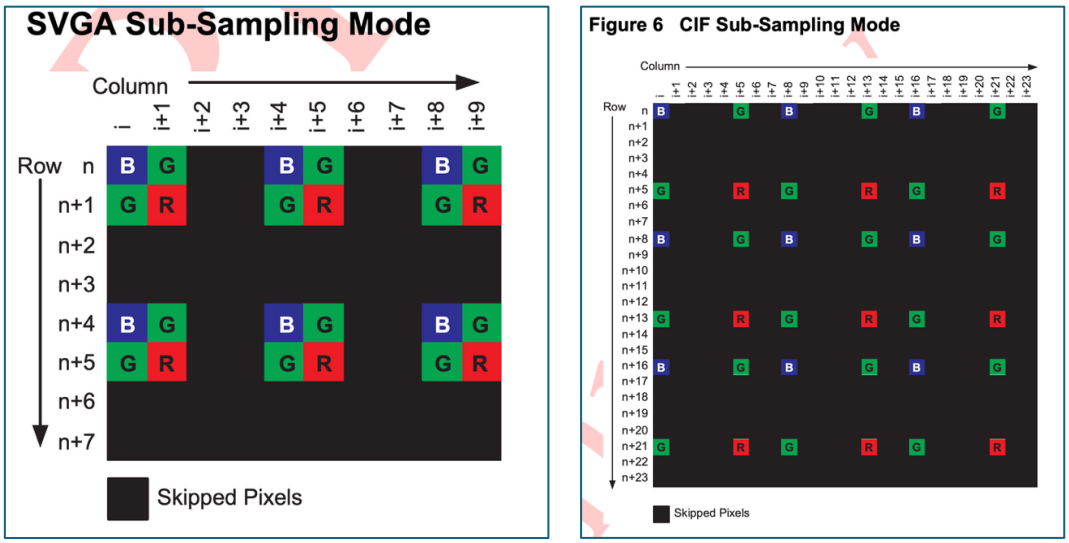


Figure 3.1 – SVGA and CIF sub-sampling modes of the OV2640 (OmniVision Technologies, 2006)

By comparing the images from the full 1600 x 1200 sensor, 800 x 600 SVGA, and 400 x 296 CIF modes, the SVGA mode seems to be the best balance between resolution and image quality. Note that there were no proper tests conducted or real-world data to back up the accuracy claim. I lack sufficient time to verify the claim, although accuracy didn't suffer greatly in my experience.

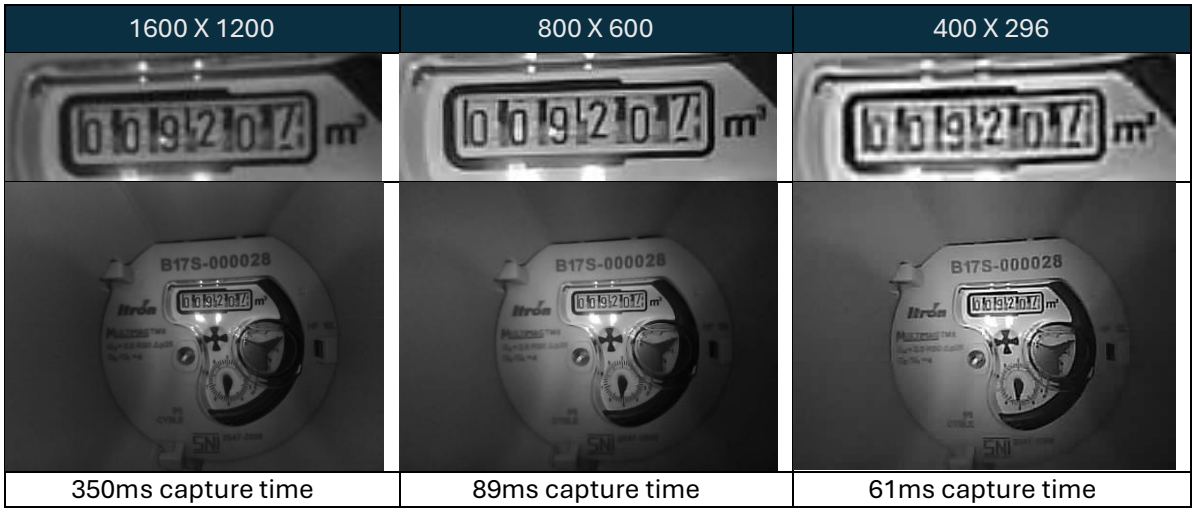


Figure 3.2 – Comparison of OV2640 capture modes: crop of the drums (top), full frame (bottom) and capture time

2. PWM dimming on the LED

I had initially dismissed the use of PWM dimming on the Capture LEDs as the OV2640 lacked a global shutter, but instead uses a rolling shutter which may be able to capture between the duty off cycle of the PWM signal from the LEDs which could cause visible banding in the output image. But I found that the OV2640 did not suffer from any banding issues, therefore I chose to run the LEDs at 20KHz with 50% duty cycle. This saves some fraction of power and adds brightness control (Figure 3.3).



Figure 3.3 – Banding vs no banding when the LEDs are PWM-dimmed

3. In camera Greyscale

The greyscale filter is done on camera by the OV2640 and as a result returns a slightly smaller image file, the impact is pretty much none to minimal, but it was easy to implement (Figure 3.4).



Figure 3.4 – In-camera greyscale vs colour capture

4. **Hardware accelerated Vector Instruction set & model quantization**

Most of the earlier builds of DrumNet ran at full precision Float32 on the ESP32-S3, this was slow and inefficient. The ESP32-S3 has built in vector instructions that can be accessed through the ESP-NN library (Espressif Systems, n.d.). It performs Multiply and Accumulate (MAC) operations multiple times each clock cycle, which allows it to speed up neural network calculations significantly, while being more efficient. After quantizing DrumNet to 8 bits, capture to prediction output time was cut from 61s to 17s a roughly ~ 3.6x increase in speed. Efficiency gains have not been properly measured, although I am sure there were big improvements there too.

3.3.2 Hardware-Based Optimizations

When dealing with low power, ultra long-lasting projects like this one, it is important to choose efficient components that are rated to run at low voltages. The wireless radios on the ESP32-S3 would only be used for initial configuration and will be turned off for the rest of its life-cycle.

1. **Low I_q and high efficiency DC-DC converters**

The power stages are the first challenge, the OV2640 itself requires 1.5V and 2.8V power to operate. I have considered using buck converters for the 1.5V and 2.8V rails as they have much better efficiency as compared to LDOs but even with the apparent inefficiencies that LDOs bring about, Buck converters have a high quiescent current and much more complex design requirements that will increase both cost and complexity while netting close to negligible gains, they may also introduce unwanted artifacting in the image if the buck converter circuit was not well designed. I was not ready to spend more time and precious board space for small efficiency gains, so my choice was to stick with LDOs and search for LDOs with low quiescent current (I_q) and reasonable efficiency to value ratios.

Although the main 3.3V rail would need to be swapped for a special low I_q (~75nA) buck boost converter by TI, the TPS63900 (Texas Instruments, 2020). As it will need to be able to handle more current (that the LoRa and MCU might need) and maintain 3.3V no matter the input voltage. It is also rated to run with Li-$SOCl_2$ batteries that I plan to use for the project.

2. **Ultra Low Power Timer interrupt ICs**

Not strictly necessary but ultra-low power timer interrupt ICs like the Ambiq AM1805 (Ambiq, n.d.) can turn off all the components and turn them back on on a recurring schedule. The efficiency gain from this would probably be minimal since most components already draw little current in deep sleep.

3. **Low Power wireless communication standards (LoRaWAN)**

LoRaWAN was chosen for its superior low power and long-range capabilities (LoRa Alliance, n.d.) that allow the reader to communicate wirelessly while on batteries. Its open standard also allows the operator to entirely own and run their network, as compared to Sigfox and NB-IoT which tap into privately owned networks.

3.4 Camera Choice

For this project, I needed a capable macro camera that is wide enough to capture the full width of the meter barrels while being in relatively close proximity to the water meter itself, there were no options that were both, low cost and of the correct connector type that were readily available from reputable retailers available for me to purchase, therefore I opted for these two cameras from Lazada instead:

- OV2640 with a 160° FOV lens (link)
- OV2640 with a 120° FOV lens (link)

They were not advertised with a minimum focusing distance or by being a macro lens, but an intrinsic property of wide angled lenses are their ultra short focusing distance. By design, wide-angled lenses have a much deeper apparent depth of field at any given distance as compared to a telephoto lens, causing more of the foreground and background to stay in focus. With that in mind, I bought these two cameras to try.

To test both cameras out, I devised a simple test, I will take an image with both cameras at the distances 3cm, 4cm, and 6cm to see how well they perform, I will be using 2 focusing charts to evaluate the performance. Chart A was hand-drawn by me, and Chart B was taken from this site by Stephen H. Westin from Cornell University (Westin, 2010; Figure 3.5).

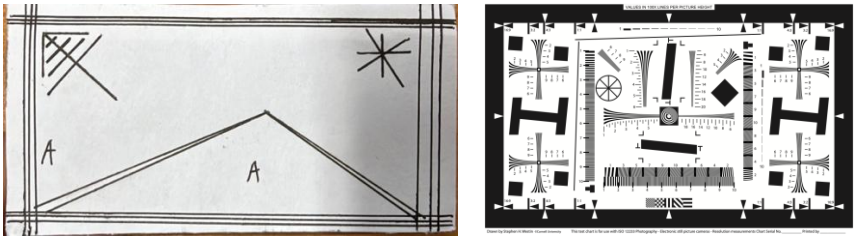


Figure 3.5 – Focus charts: Chart A (left, hand-drawn) and Chart B (right; Westin, 2010)

Using a spare ESP32-S3 cam board, with the help of an AI agent, I was able to re-program the board with a simple web applet that allowed me to setup the camera and snap a picture of whatever was in view, the camera and the board was mounted to the side of the box and a ruler was placed starting from where the camera lens ended. Then the charts were mounted on a moveable stand that allowed the chart to be placed at a set distance away from the camera (Figure 3.6).

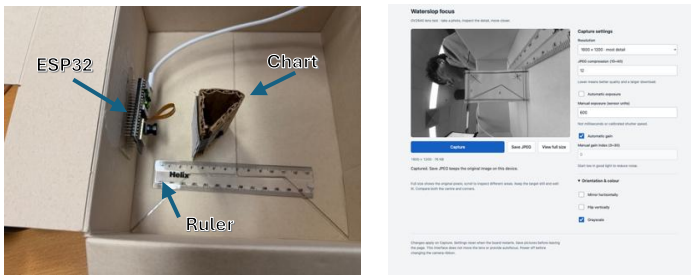


Figure 3.6 – Focus test setup (left) and the camera web applet (right)

Figure 3.7 and Figure 3.8 show the focus charts at each distance for each lens.

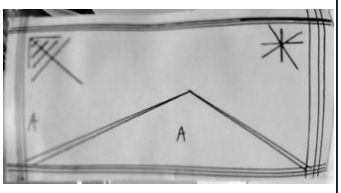
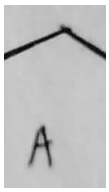
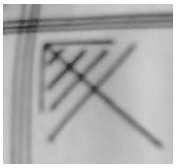
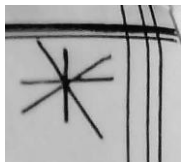
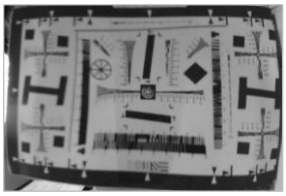
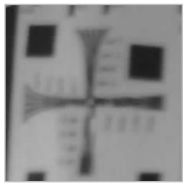
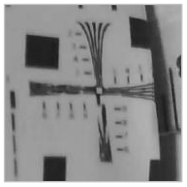
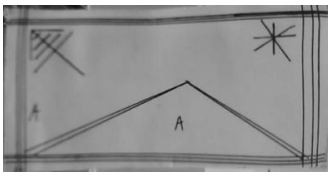
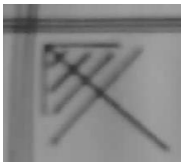
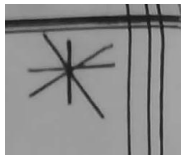
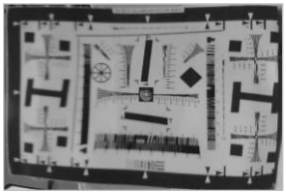
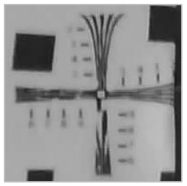
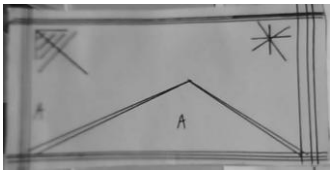
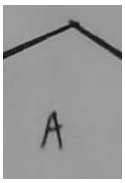
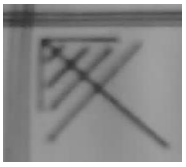
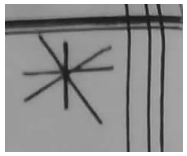
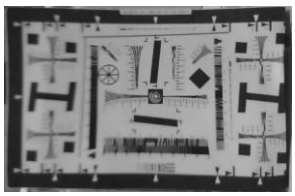
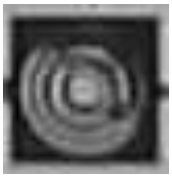
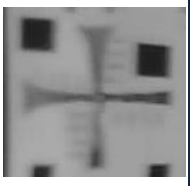
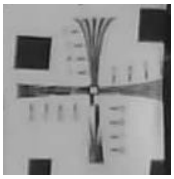
Distance	Chart	Full frame	Centre	Left	Right
3 cm	A				
	B				
4 cm	A				
	B				
6 cm	A				
	B				

Figure 3.7 – Focus test results for the OV2640 with the 160° FOV lens

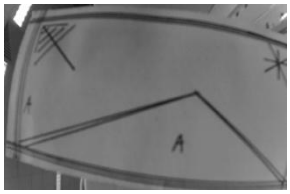
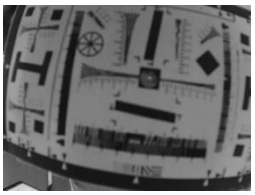
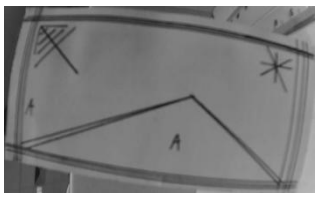
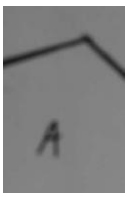
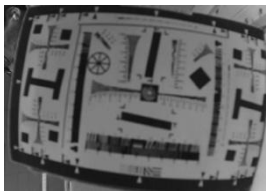
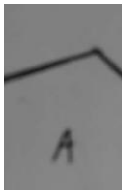
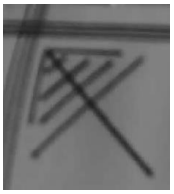
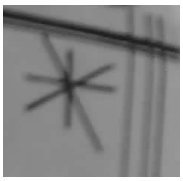
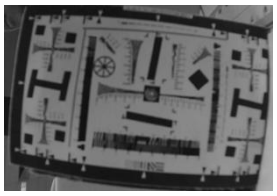
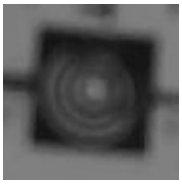
Distance	Chart	Full frame	Centre	Left	Right
3 cm	A				
	B				Not In frame
4 cm	A				
	B				
6 cm	A				
	B				

Figure 3.8 – Focus test results for the OV2640 with the 120° FOV lens

Based on the results of the test conducted above, it is very clear that the 120° FOV variant of the OV2640 is unable to focus well on close objects (macro). I would also like to mention that these are fixed focus lenses, so focus is permanently set at the factory. Moving forward I will be using the 160° FOV variant at 4 cm for this project (Figure 3.9).

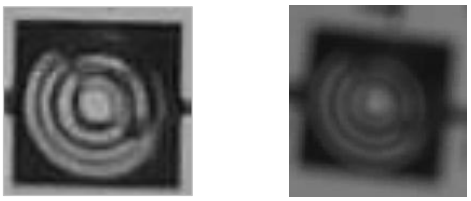
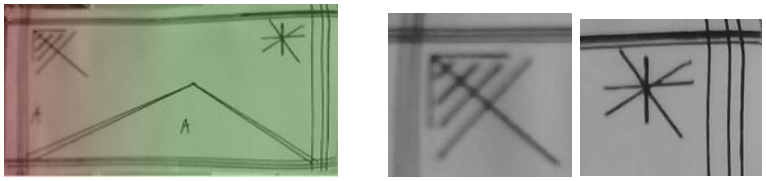


Figure 3.9 – Chart B at 4 cm: 160° lens (left) and 120° lens (right)

I would also like to mention a finding from the results, on the 160° FOV variant, focus seems to taper off towards the left of the image from the center, while this focus "tapering" effect was nowhere near as strong in the opposite direction. I have also ensured that the paper was positioned perpendicularly to the camera and there were no major slants or dents that may have caused this effect. This effect was however not present in the 120° variant, it had a conical and even focus tapering, that is expected. This points towards a manufacturing defect, but I do not have any extra cameras to test if that is true. Figure 3.10 illustrates the effect.



Bad Focus Good Focus Left Corner Right Corner

Figure 3.10 – Focus tapering effect of the 160° lens

3.5 Prototype V1

Prototype V0.5 (ESP32-EYE) lacked easily accessible GPIO pins for secondary development, thus Prototype V1 was created, it added a LoRa radio, LED pins and limit switches. A M5Stack Cardputer LoRa cap was repurposed for its LoRa radio. A perfboard was used to mount all components together. Prototype V1 was primarily used by the test rig to capture images for training and also for evaluating model performance (Figure 3.11).

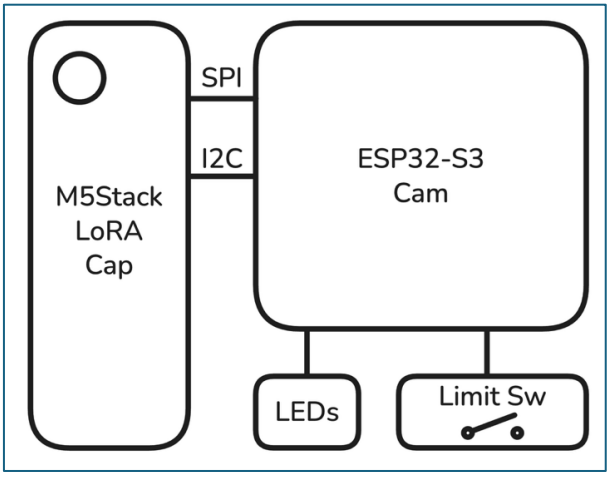


Figure 3.11 – Prototype V1 block diagram

Prototype V1 was mounted onto the V1 enclosure, it was designed to be fitted above a water meter but was instead used as part of the test rig to improve model performance (Figure 3.12).

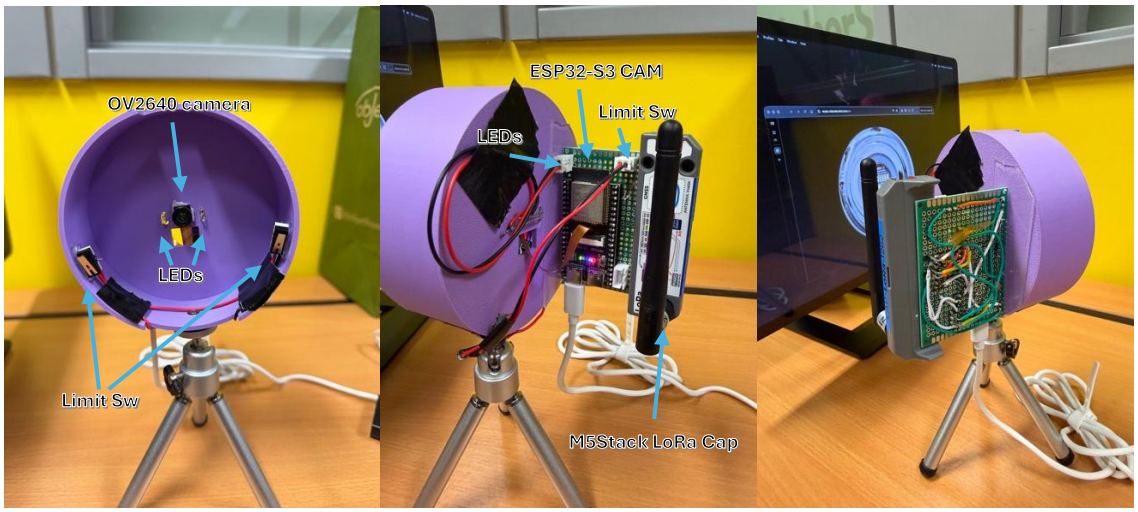


Figure 3.12 – Prototype V1 assembled

3.6 Prototype V2

Prototype V2 is a shrunken down version of Prototype V1, it allows me to fit it in tighter spaces and explore smaller hardware components like the Wio-SX1262 LoRa module from Seeed Studio (Seeed Studio, 2024).

Note that Prototype V2 was developed before the writing of this report, as such some deprecated decisions are still part of this prototype. Parts like the fan, which was proposed to be used during high humidity conditions that may cause fogging. If the readings are not reliable, it will be dropped instead. The power circuitry also pre-dates the decision of using Li-$SOCl_2$ cells and reflects as such in the design.

3.6.1 Planning and Layout

Prototype V2 has built in battery management, power circuitry for the camera and fan (deprecated), PWM dimmable LED lights and finally a LoRa module for communication.

Primary components:

- **ESP32-S3 WROOM1 N16R8** chosen MCU for this project
- **WIO-SX1262** All in one LoRa module from Seeed Studio (SPI)
- **SHT-40** Humidity and temperature sensor (I2C)

LiPo Battery protection and charging components:

- **BQ24074RGTR** Battery charging IC
- **BQ29700DSER** Battery protection IC
- **MAX17048G+T10** Battery monitoring IC (I2C)

Other Active components:

- **MIC5365-2.8** **2.8V** linear LDO Regulator (camera)
- **TPS7A0315PDBVR** **1.5V** linear LDO Regulator (camera)
- **TPS63021DSJR** **3.3V** Buck-Boost converter (general)
- **TPS61023DRLR** **5V** Boost converter (fan – deprecated)
- **AO3400A** N-Ch MOSFET
- **USBLC6-2SC6Y** USB ESD protection

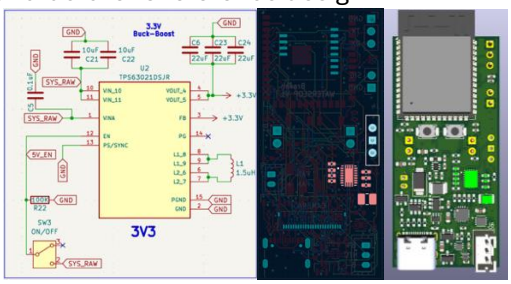
Other Passive components:

- Resistors ranging from 22R to 732K, 0402 and 0603 sizes
- Capacitors ranging from 0.1uF to 22uF, 0402 and 0603 sizes
- Inductors 1uH and 1.5uH at 1210 size
- Push buttons and slide switches
- SS14 Schottky diode (for the deprecated fan)
- Flat top LEDs
- USBC connector

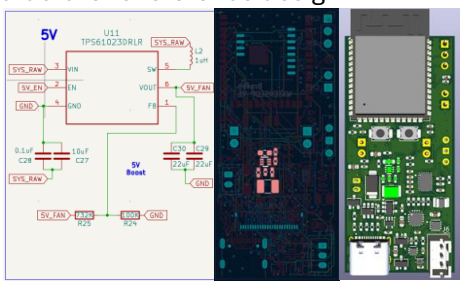
3.6.2 Power Circuit

Next I started on the schematic of the PCB, I used KiCad (KiCad, n.d.) to design the board. First was designing the DC-DC conversion power circuitry to get 5V (for the deprecated fan), 3.3V for the MCU, LoRa module and camera, then 1.5V and 2.8V for the camera itself. Finally the battery charging monitoring and protection circuitry. A Buck-boost converter was used for the 3.3V rail as it was connected to the battery and voltage could fluctuate above or below 3.3V based on the state of charge of the battery, the 1.5V and 2.8V LDOs were chosen for their low noise as it is used to power the cameras, and a Boost converter was used for the deprecated fan as it draws significant power and could benefit from the efficiency of the boost converter (Figure 3.13).

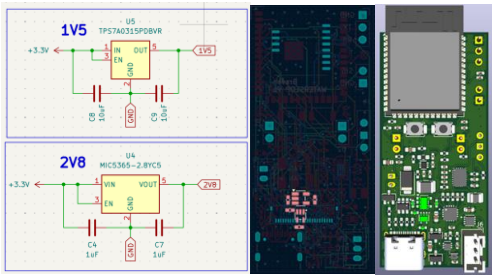
3.3V Buck-Boost Circuit
The schematic was mostly copied from the manufacturer's reference design.



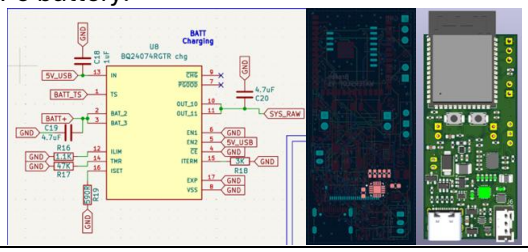
5V Boost Circuit
The schematic was also mostly copied from the manufacturer's reference design.



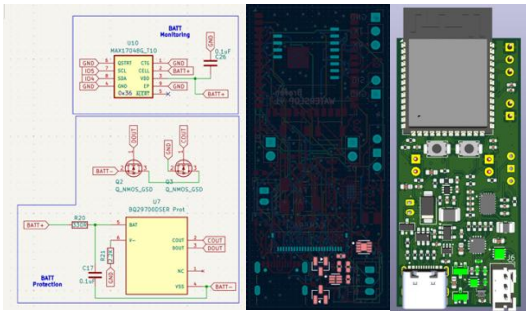
1.5V & 2.8V LDO Reg Circuit
Chosen for simplicity and that they produce no noise as compared to buck converters.



LiPo Battery Charging Circuit
Handles power switching from USB and battery, its output feeds into the 3.3V Buck-Boost chip. It manages charging of the built in LiPo battery.



Battery Protection & Monitoring Circuit
The BQ29700 protects the LiPo cell from over-charge, over-discharge and over-current; the MAX17048 fuel gauge reports its state of charge over I^2C.



Full Power circuitry
All power circuits combined. They take up ~ 40% of board space.

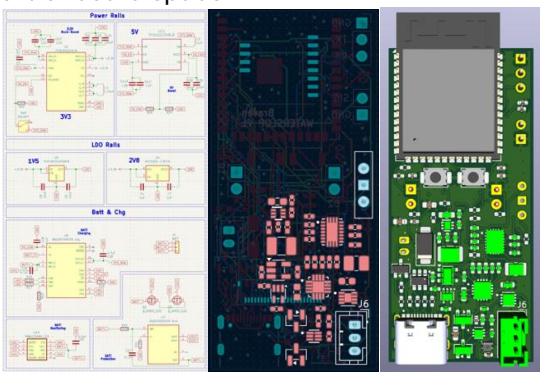
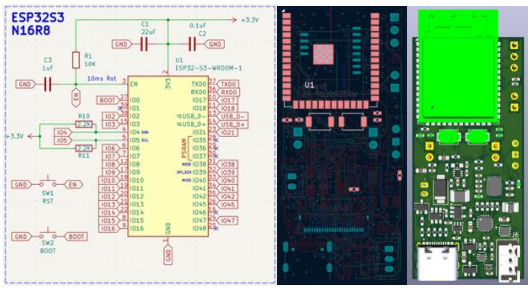


Figure 3.13 – Power circuits: schematic (left) and location on the board (right)

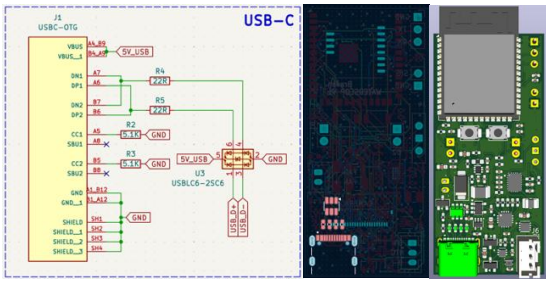
3.6.3 Peripherals and MCU

Figure 3.14 shows the MCU and the peripheral circuits on the board.

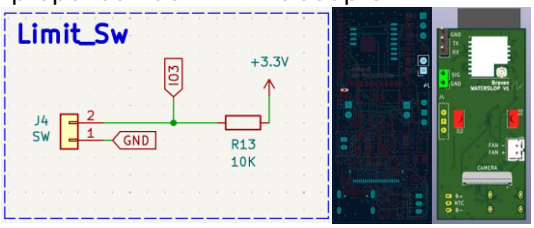
ESP32-S3 MCU
The MCU controls all the peripherals on this board and runs the neural network.



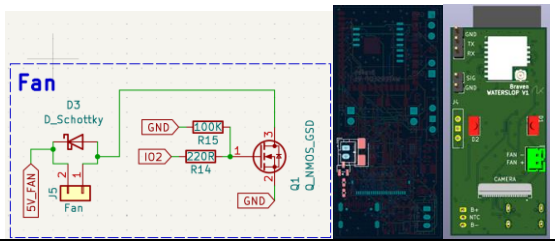
USBC & ESD Protection
Connected to the ESP32's USB OTG interface, a diode array chip is added to protect against ESD.



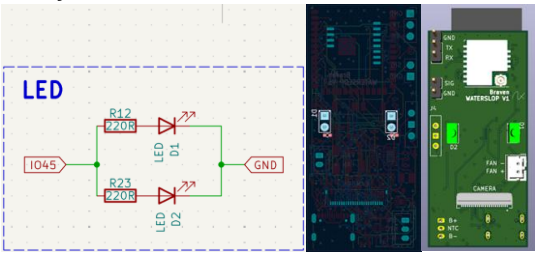
Limit Switches
mounted to the edges of the cap to detect improper contact with the adapter.



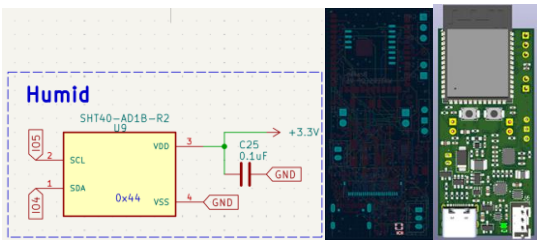
Fan Control circuit
Uses a N-Ch MOSFET to control the fan, powered from the 5V Boost circuit



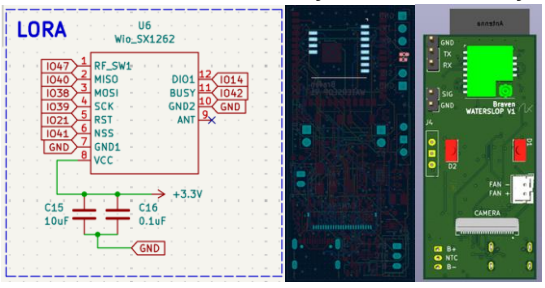
PWM controlled LEDs
They are flat-top LEDs unlike the ones portrayed in the visualization



I²C Humidity and Temperature sensor
Used to measure the environmental condition around the reader, skips a read in bad conditions



LoRa Module
Low power long range communication that allows the reader to run for years on a battery.



24 Pin connector for DVP Cameras
Allows the camera to be connected to the board.

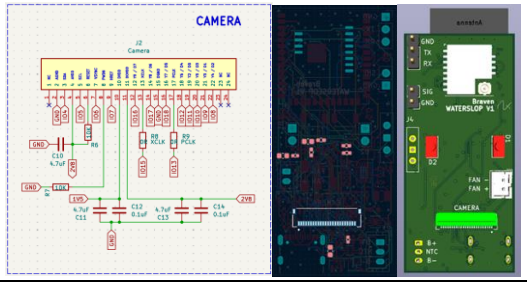


Figure 3.14 – MCU and peripheral circuits: schematic (left) and location on the board (right)

3.6.4 PCB Layout and Manufacturing

Finally, I sourced the components from DigiKey and sent the PCB to JLCPCB (JLCPCB, n.d.) for manufacturing; it is a 4-layer board (Figure 3.15 and Figure 3.16).

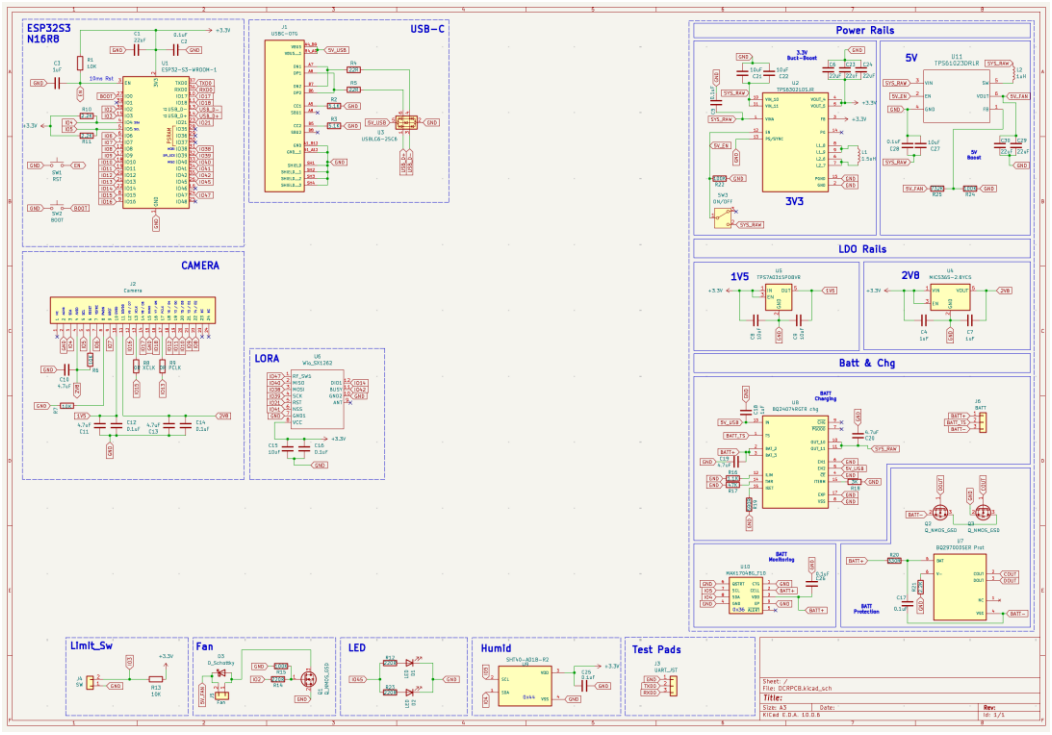


Figure 3.15 – Full board schematic from KiCad

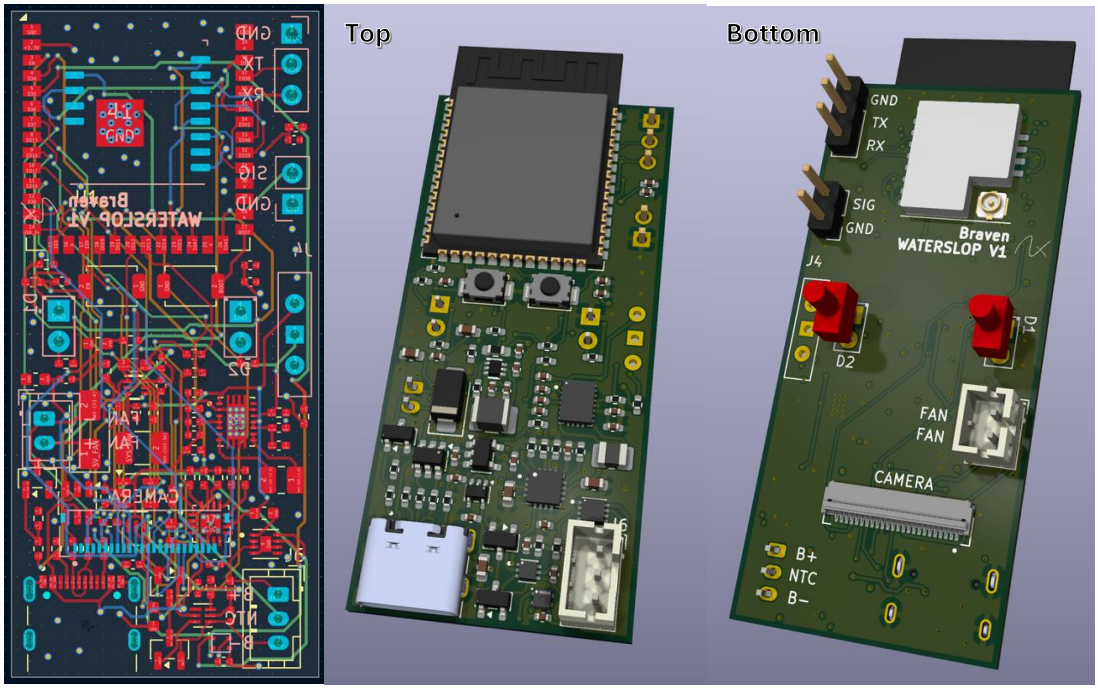


Figure 3.16 – PCB layout and 3D renders of the board from KiCad

The PCB was designed to be roughly the same size as a typical ESP32-S3 dev board, this compact design allows me to create a smaller housing that would fit meters in tighter spaces (Figure 3.17).

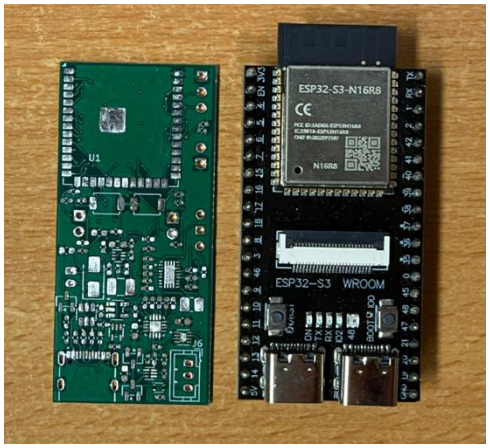


Figure 3.17 – Prototype V2 PCB next to an ESP32-S3 development board

3.6.5 PCB Assembly

First was using a stencil to spread solder paste onto the pads then placing the components onto the board and reflowing the solder on a hotplate (Figure 3.18). Unfortunately as I was preparing to solder the components, I found out that I had forgotten to add the battery protection IC chip to my order therefore I opted to not use the battery for this part of the project, this was the first out of the 3 problems I faced.

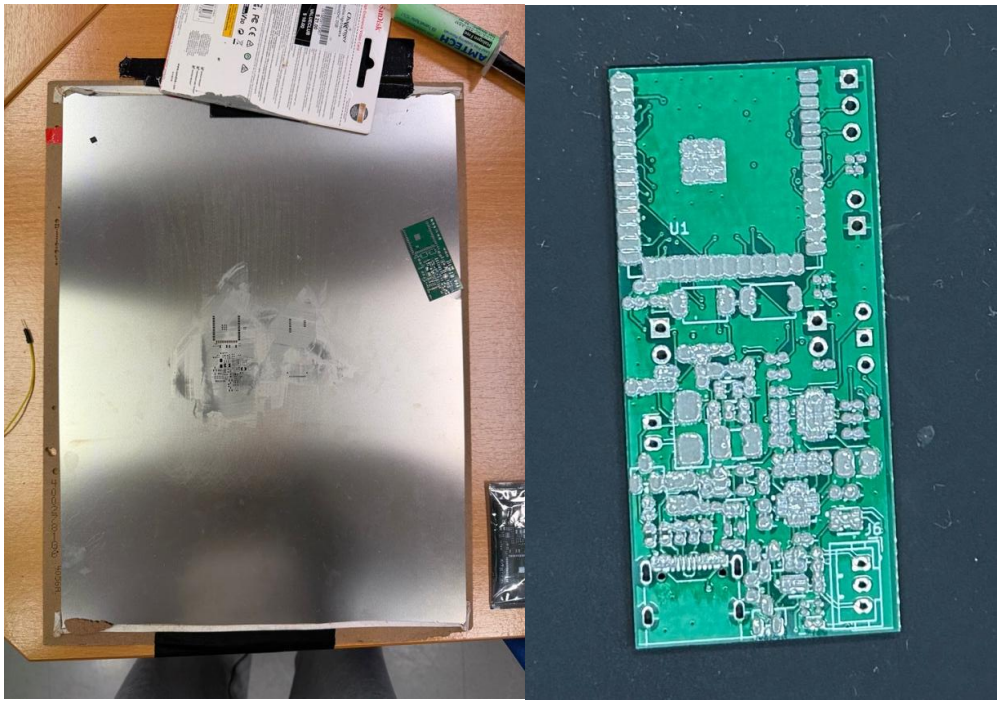


Figure 3.18 – The solder stencil (left) and the board with solder paste applied (right)

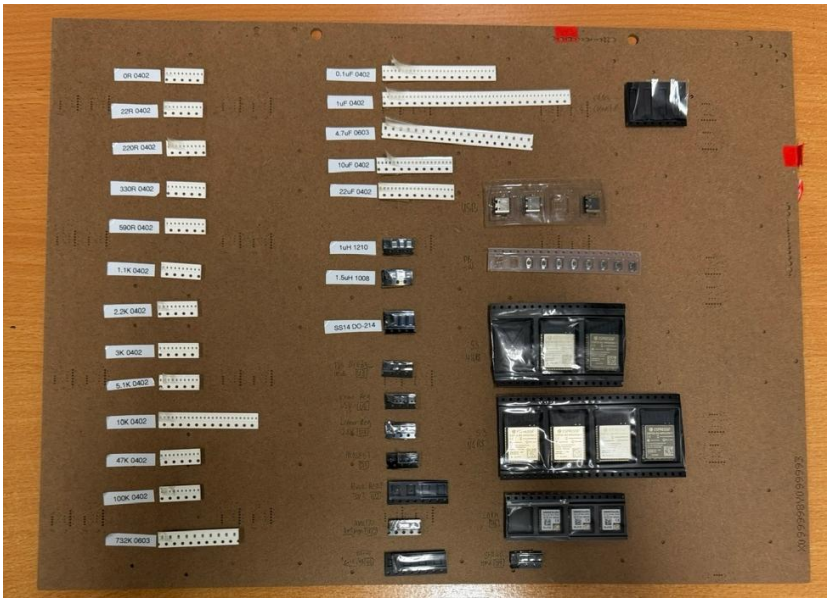


Figure 3.19 – Components laid out for assembly

It took about 9 hours to fully assemble the board without any visually noticeable defects, then I stripped a USBC cable to connect it to my benchtop power supply. I set my benchtop power supply to 5V with 400mA current limit. Almost immediately after I turned my power supply on, I heard the current limit alarm beeping, the current limiter was doing its job and the board was drawing the maximum 400mA and voltage dropped to 4.2V, after which I promptly cut off power to the board. It was not the expected behavior from this board, so I started investigating (Figure 3.19 and Figure 3.20).

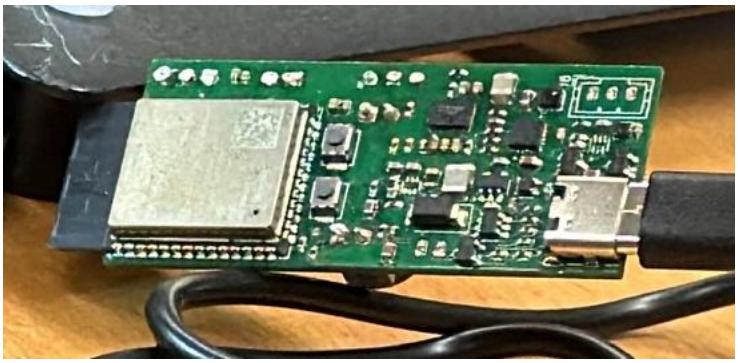


Figure 3.20 – The assembled board on the benchtop power supply

Next I tried raising the current to 600mA and applied power to the board and see if it hits the current limit too, 600mA would be an unreasonable amount of current for this board to be drawing under normal circumstances, keep in mind that the fan was not plugged in as I have decided on not using it by this point, so there are no high current drawing component on the board. When I turned the power on, it immediately hit 600mA. Now that I confirmed that there had to be a short somewhere on the board, I started hunting for it, first I tried using my multimeter in continuity mode to try to find which stage of the power conversion had a short, but due to how small the components were, it was extremely hard to definitively probe at the pins.

Then I placed my finger over the whole board and turned on the power supply to feel what got hot, the battery charging IC got very hot in roughly 2 seconds after power was applied, so I quickly turned the power off and inspected the chip. I initially suspected that the pins beneath the chip could be bridged, so I desoldered the chip with a hot air gun and used a solder wick to wick away as much solder as I could, then placed fresh solder paste on and melted it, ensuring that they looked good before placing

a new battery charging IC onto the board (just to be safe). Turning the power supply back on revealed the same problem as before.

The next downstream component of the battery charging IC was the 3.3V Buck-Boost chip which was my next suspect in hunting down the short. First I tried reflowing the chip with my hot air gun, which produced the same results, I then inspected the solder connection of the chip closely under a microscope (Figure 3.21).

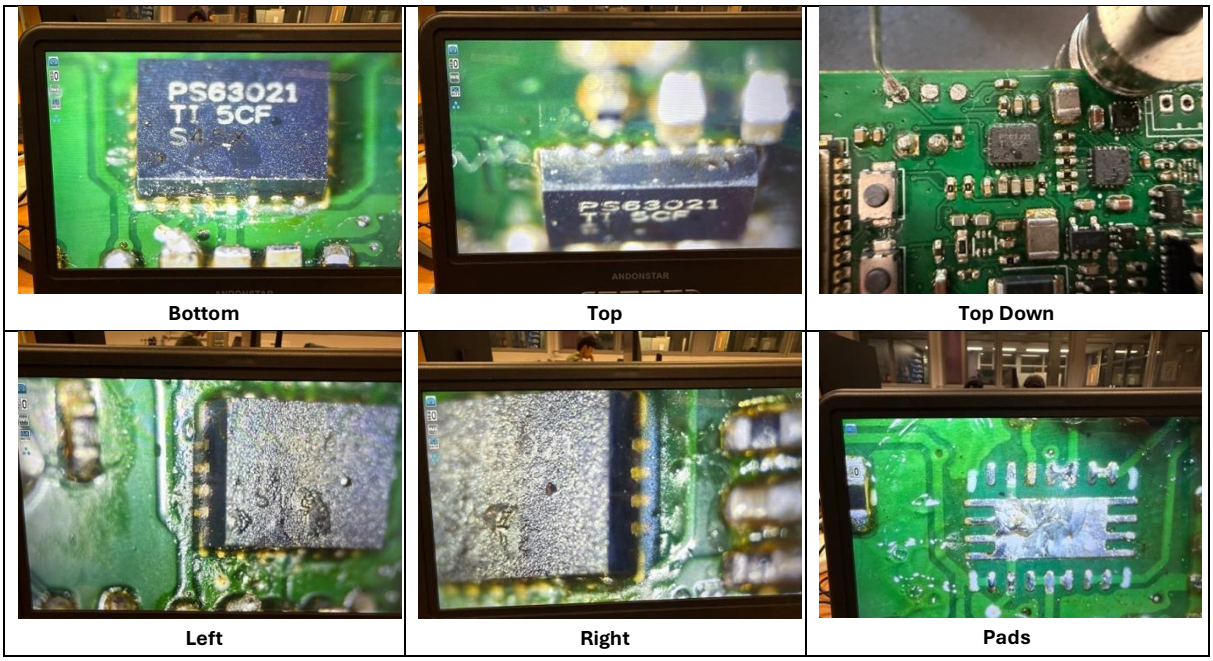


Figure 3.21 – Microscope views of the 3.3 V buck-boost chip

I then also replaced the chip with a fresh one which removed the short but did not output any voltage at all, after which I noticed that the enable pin pad of the footprint was lifted and broken, it must have failed from the many reflow attempts of me replacing chips, during that process the double-sided tape that held the camera to the board failed so I removed the camera (key to this problem). Since there was no saving this circuit anymore, I opted to try injecting 3.3V from my power supply externally, I tapped thin enamel wires to the 3.3V rail of the board and powered it from my power supply. This time the short went away and was drawing 70mA at 3.3V which was well within the expected range. Then I held down the reset button of the ESP32-S3 which stop execution on the chip, the current expectedly dropped down to 50mA which shows that the ESP32-S3 chip was working as expected.

Next I connected a USB cable from my computer to the board, and it was detected successfully which was a relief. Which then, I asked an agent to write a simple blinky program to be flashed onto the board. At this point I figured that powering the board from my benchtop power supply was too clunky, so I decided to scrap an old esp32 board to use its proven working regulators to convert the 5V from the USB input into 3.3V to power my board, thus bypassing the 3.3V Buck-boost circuit entirely (Figure 3.22).

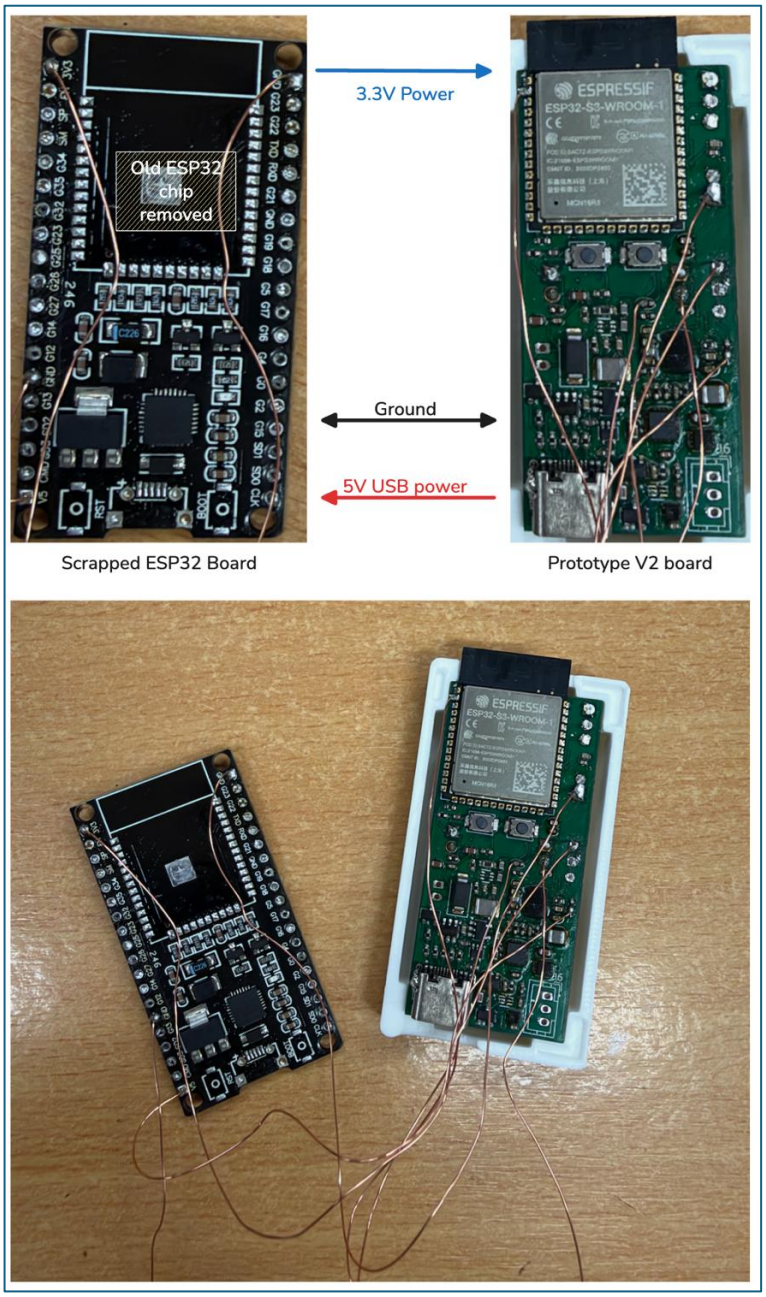


Figure 3.22 – 3.3 V bypass using the regulator of a scrapped ESP32 board

Now that I have confirmed that the board could power on and work correctly, I reconnected the camera back onto the board and to my surprise, the current limit was triggered on my power supply yet again. That was when I realized that the camera was the actual culprit causing the short, I replaced the current camera that was plugged in with another camera which produced the same results yet again, that led me to suspect that the FPC ribbon connector was not correctly soldered into place, or there could be a joint that was shorted. I tried replacing the connector and experimenting with different amounts of solder on the pads, but the connectors were very fragile and could not survive the heat from my hot air gun, and I had broken 3 of them. While I was working with my last connector, I noticed that the camera connector was flipped around and incorrect. The first pins did not line up with the connector in the normal orientation that the camera is mounted, instead it was on the opposite end. The only way I could rectify this error was to flip the camera around instead (Figure 3.23).

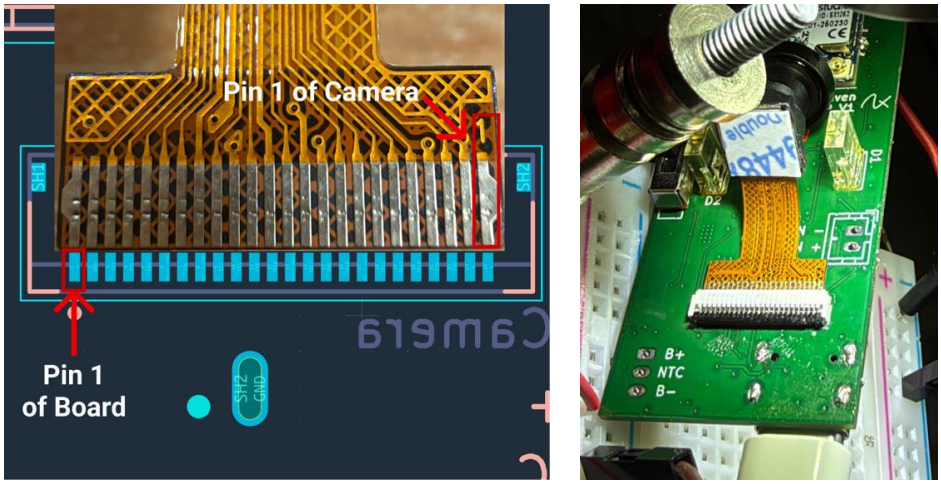


Figure 3.23 – Mirrored camera pin-out (left) and the flipped camera (right)

After flipping the camera around, the board booted up fine and the camera was correctly detected, However when I went to attempt to take a photo with the camera, I encountered yet another error, a “Camera Frame Timeout” error. I started by testing the connectivity of PCLK and XCLK with my multimeter as they were responsible for timing. I found that PCLK had no connection from the connector of the camera to the MCU (Figure 3.24).

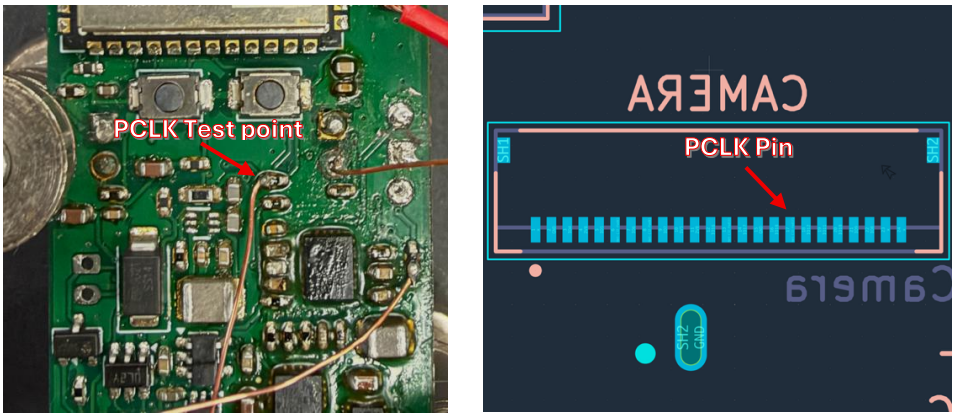


Figure 3.24 – PCLK test point (left) and PCLK pin at the camera connector (right)

After some attempts, and with help from Mr Soh, the connector was soldered successfully. After which the camera was able to take images without problems, and I moved on to the next stage of the project (Figure 3.25).



Figure 3.25 – First successful image capture on Prototype V2

In hindsight, the camera, not the 3.3V buck-boost converter, was most likely causing the short. I should have assembled the board in stages and only moving on to the next stage when the previous stage is confirmed to be working well. The 3.3V circuit would not have been damaged if not for that mistake, but due to time constraints I had to rush.

3.6.6 Interior Enclosure

The interior enclosure was designed to house the PCB and batteries of Prototype V2. I started by exporting the 3D file from KiCad and importing it into Onshape (Onshape, n.d.), where I will be using the model from KiCad as a reference to design the enclosure around. Next I quickly modeled the exact shape and size of the camera in Onshape to be added to the reference design for better accuracy (Figure 3.26).

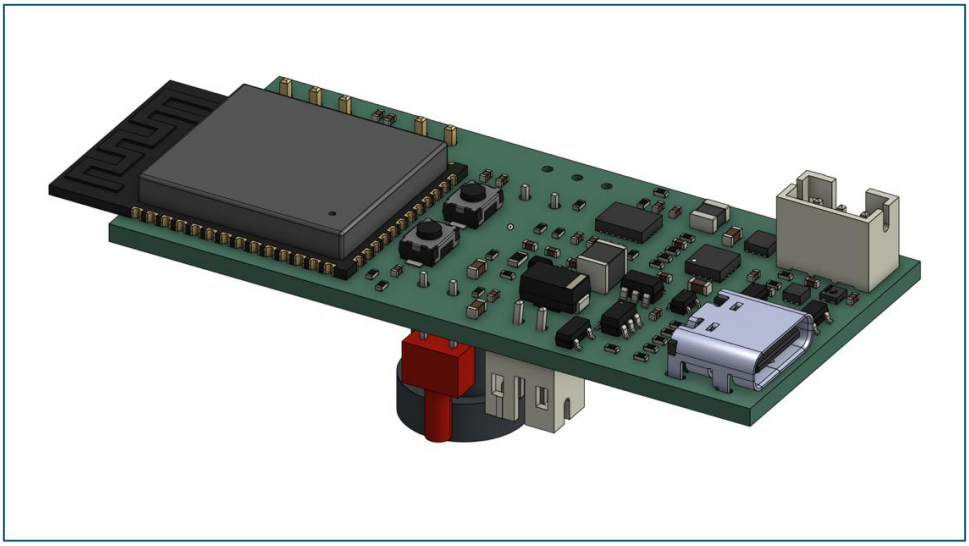
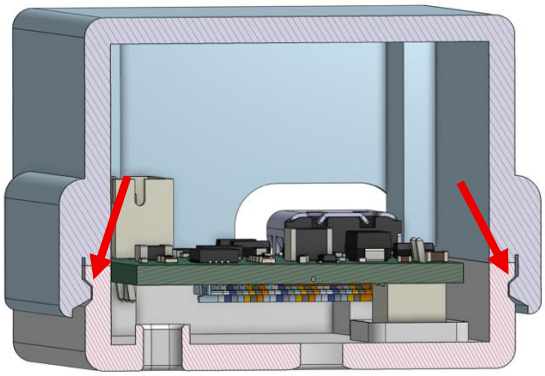


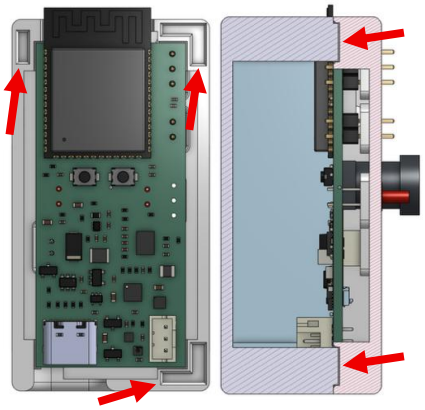
Figure 3.26 – KiCad 3D model used as the reference for the enclosure design

Figure 3.27 shows some notable parts of the final enclosure design. It adopts a toolless design so no screws or tools are needed to assemble or disassemble the enclosure.

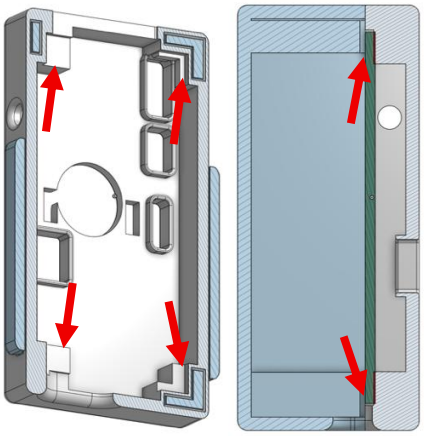
Side Clips
The side clips allow the top and bottom half of the enclosure to snap on with no screws needed.



Lateral movement stopper
Groves are cutout from the bottom half to where pins from above are inset, to prevent lateral slide.



PCB holder
Platforms are added on the bottom of the case and posts from above hold the PCB to the case without any screws needed.



LoRa Antenna Cutout
A special hole is cutout for the LoRa IPEX cable to interface with the LoRa transceiver on the board.

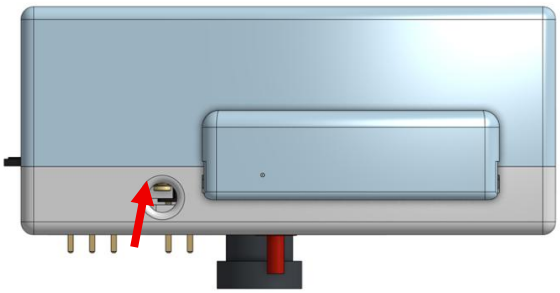


Figure 3.27 – Notable features of the interior enclosure

The fit and finish of the final enclosure was great, but I had to cut a hole to fit the flipped camera from the incorrect pin layout error of the PCB assembly stage (Figure 3.28).





Figure 3.28 – Final interior enclosure (left) and earlier revisions (right)

3.6.7 Exterior Enclosure (Meter Adapter)

I edited my original water meter adapter so that the camera will be positioned ~4 cm away from the meter (Figure 3.29 and Figure 3.30). Link to model here

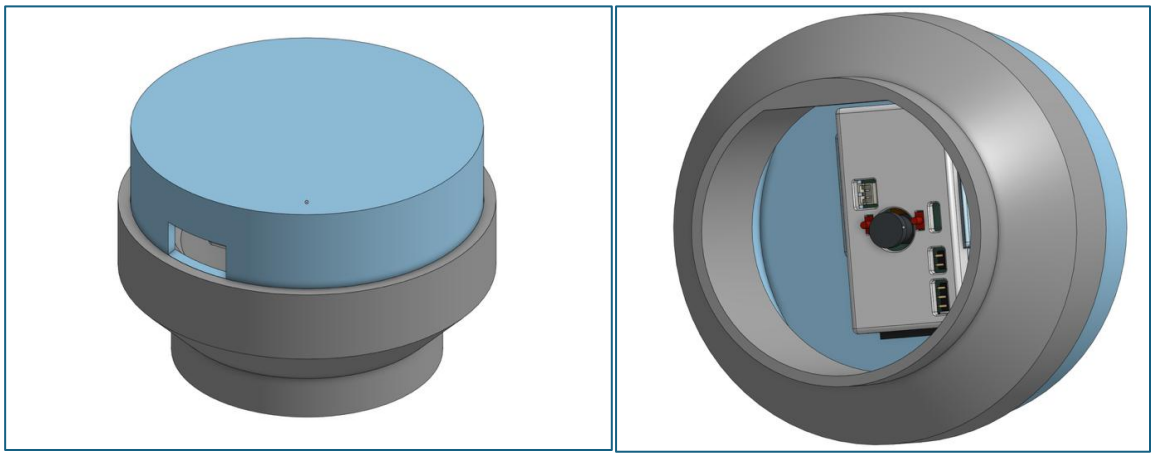


Figure 3.29 – Meter adapter: top view (left) and bottom view (right)

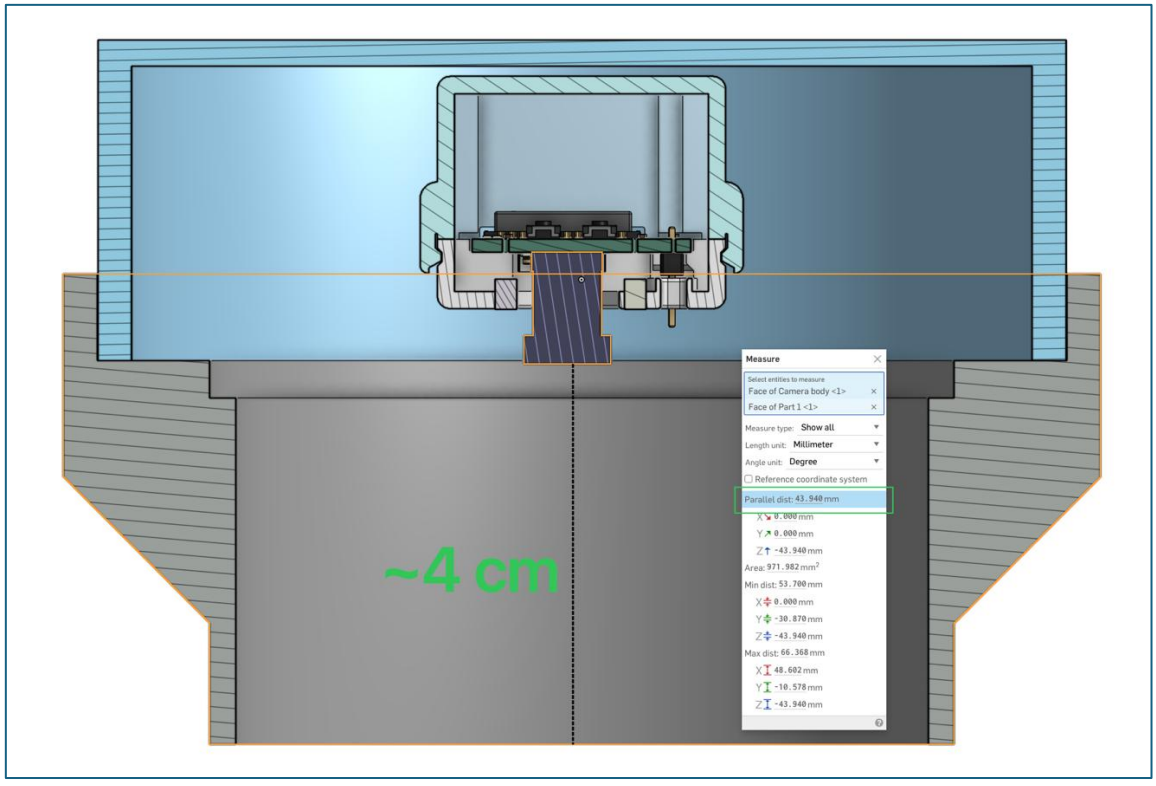


Figure 3.30 – Distance from the camera lens to the meter face (about 4 cm)

The adapter is separated into two parts, the top half and the bottom half. The bottom half is mounted against the meter as shown in Figure 3.31. While modeling, I had forgotten to take into account the tabs on the side of the meter that allows the lid to be securely locked in place, this caused the bottom half to be unable to be mounted flush against the meter face. Luckily the bottom half was printed with the layer lines parallel to the meter face, so I was able to make a few simple cuts with a pair of side cutters for the tabs to sit in.

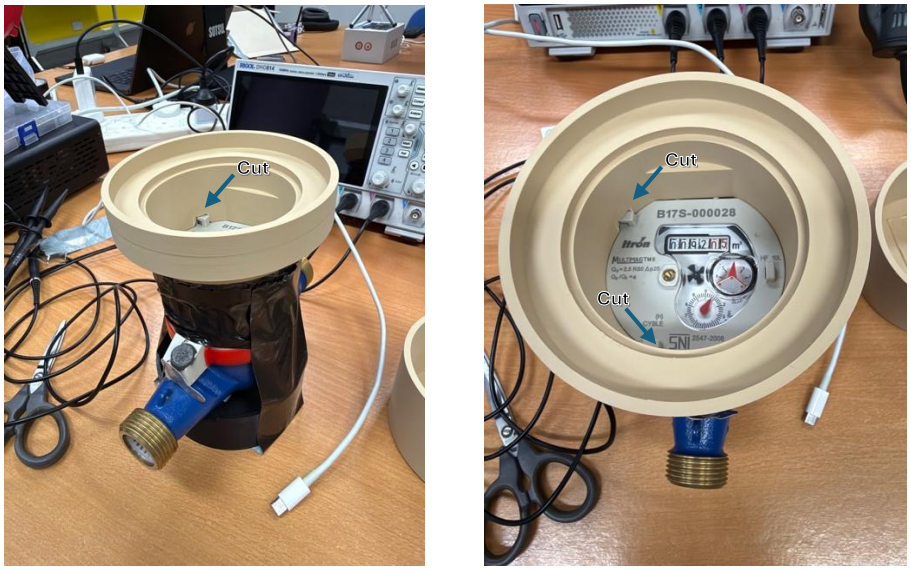


Figure 3.31 – Cuts made so the adapter sits flush against the meter

Next was the Top half of the adapter, it was designed with a much larger diameter as compared to the Bottom half. This creates more space for Prototype V2's position to be adjusted for a better alignment to the meter barrels. It also allows me to reuse the Top half with other meters just by redesigning a Bottom half adapter for each new meter and transporting the Top half between them (Figure 3.32).

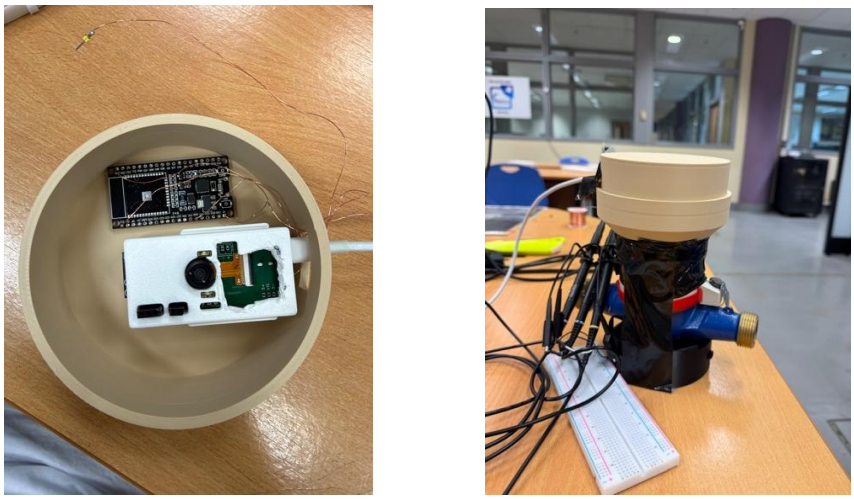


Figure 3.32 – The completed adapter

3.6.8 Camera Calibration

With the help of an AI agent, I ran a test to find out how each camera setting and parameter affect the image taken, the goal was to get a clean and acceptable image while minimizing LED On time and reducing capture time. Previously in the software optimization section, 800 x 600 SVGA sub-sampling was identified as the preferred resolution to capture in. The test included exposure, gain, PWM duty-cycle and frequency, pulse duration, contrast, raw gamma and LED start delay. Table 3.4 summarises the results.

Table 3.4 – Camera settings and their observed image effects

Factor and observed effect	Example 1	Example 2	Example 3 / limitation
Exposure Tested: 84, 168, 336, 448, 672, 1000 No PWM 84 was too dark. 336 brightened the digits. Higher exposure gave variable brightness rather than a consistent improvement.	**84**  Digit mean 33.2/255 Contrast 43	**336**  Digit mean 93.1/255 Contrast 134	**1000**  Digit mean 131.4/255 Contrast 160
Gain Tested: Indices 0, 1, 2, 3, 4, 5, 6, 8 No PWM Increasing gain brightened the image, but clipped highlights at high settings. At low PWM duty, higher gain also reduced visible detail.	**Index 0**  Digit mean 93.1/255 Contrast 134	**Index 2**  Digit mean 167.0/255 Contrast 160	**Index 4**  Digit mean 212.8/255 Contrast 117
PWM duty Tested: 10, 15, 20, 25, 35, 50, 75% 20Khz PWM Lower duty reduced illumination and estimated cumulative LED on-time. Exposure or gain compensation was needed to regain brightness.	**25%**  Digit mean 33.6/255 Contrast 40	**50%**  Digit mean 58.0/255 Contrast 87	**75%**  Digit mean 76.7/255 Contrast 115
Pulse duration Tested: 20, 40, 60, 80, 100, 150 ms No PWM Short pulses caused partial lighting or dark images. The same 60 ms setting gave digit means from 5.7 to 94.3, so a single good shot was misleading.	**20 ms**  Digit mean 6.0/255 Contrast 2	**60 ms**  Digit mean 5.7/255 Contrast 2	**150 ms maximum**  Digit mean 96.9/255 Contrast 136

Factor and observed effect	Example 1	Example 2	Example 3 / limitation
PWM frequency Tested: 10, 20, 40 kHz PWM All three produced readable digits. These small samples did not establish a clear detail advantage; 20 kHz was retained.	**10 kHz**  Digit mean 99.0/255 Contrast 101	**20 kHz**  Digit mean 92.2/255 Contrast 101	**40 kHz**  Digit mean 97.1/255 Contrast 100
Contrast Retested: 0, +1, +2 Exposure 336; gain 4; 25% PWM at 20 kHz; brightness 0; gamma on. Median digit contrast rose from 104 to 117 to 129. Median brightness stayed about 106–108. Higher contrast improved tone separation while preserving grayscale.	**0**  Digit mean 107.9/255 Contrast 100	**+1**  Digit mean 105.5/255 Contrast 115	**+2**  Digit mean 106.5/255 Contrast 126
Raw gamma Retested: On / off Exposure 336; gain 3; 25% PWM at 20 kHz; brightness and contrast 0. Turning gamma off reduced median digit brightness from 93 to 36 and contrast from 105 to 46. Gamma on retained more visible shadow detail.	**On**  Digit mean 93.6/255 Contrast 105	**Off**  Digit mean 35.8/255 Contrast 45	**Retest method** Three passes in different orders; 12 settled captures per level. Only the named setting changed within each comparison. All 165 retest images had identical red, green and blue channels.
Brightness Retested: −1, 0, +1 Exposure 336; gain 2; 35% PWM at 20 kHz; contrast 0; gamma on. Median brightness was 144, 96 and 176 respectively. Both nonzero levels brightened this grayscale pipeline; −1 did not darken it. These control values are not a linear exposure scale.	**−1**  Digit mean 144.0/255 Contrast 122	**0**  Digit mean 96.6/255 Contrast 122	**+1**  Digit mean 176.8/255 Contrast 123

Factor and observed effect	Example 1	Example 2	Example 3 / limitation
LED start delay Retested: 0, 10, 20 ms Exposure 336; gain 2; 50% PWM at 20 kHz. Timed mode; 1000 ms duration limit for every level, never reached. Median brightness stayed near 120. At 20 ms delay, estimated LED on-time fell from 62 to 47 ms, but the darkest repeat fell to 81 (0 ms minimum: 114). Zero delay was more consistent.	**0 ms: typical**  Digit mean 118.8/255 Contrast 135	**10 ms: typical**  Digit mean 119.7/255 Contrast 137	**20 ms: darkest repeat**  Digit mean 80.8/255 Contrast 108

Notes: Digit mean is the average brightness in the digit region; contrast is its 90th–10th percentile intensity range. Images retain their original brightness. Typical examples are individual captures near the group median; the pulse-duration row includes the darkest 60 ms repeat. Contrast, raw gamma, brightness and LED delay were retested after correcting the shared grayscale and tone controls. Each level had three passes of five captures; the first capture of each block was retained but excluded from the 12-sample summaries. All 165 retest images were verified grayscale. The 20 ms delay image shows its darkest settled repeat. LED on-time is estimated from the measured illumination window and PWM duty, not measured electrical energy. Results apply to this fixed scene; OCR accuracy was not tested.

3.6.9 Setup and Configuration (ConfigKing)

DrumNet and the firmware are loaded onto Prototype V2, The LoRa device address will be baked into the firmware during flashing. The setup is done using an M5Stack Cardputer over BLE and ESPNOW. The M5Stack Cardputer is flashed with a piece of firmware called “ConfigKing”, ConfigKing automatically detects nearby unconfigured readers and prompts the user to set it up. ConfigKing also allows users to reconfigure configured readers, since the readers are mostly in sleep state, pressing the boot button wakes it up to be discoverable by ConfigKing. Figure 3.33 shows the configuration flow of the reader.

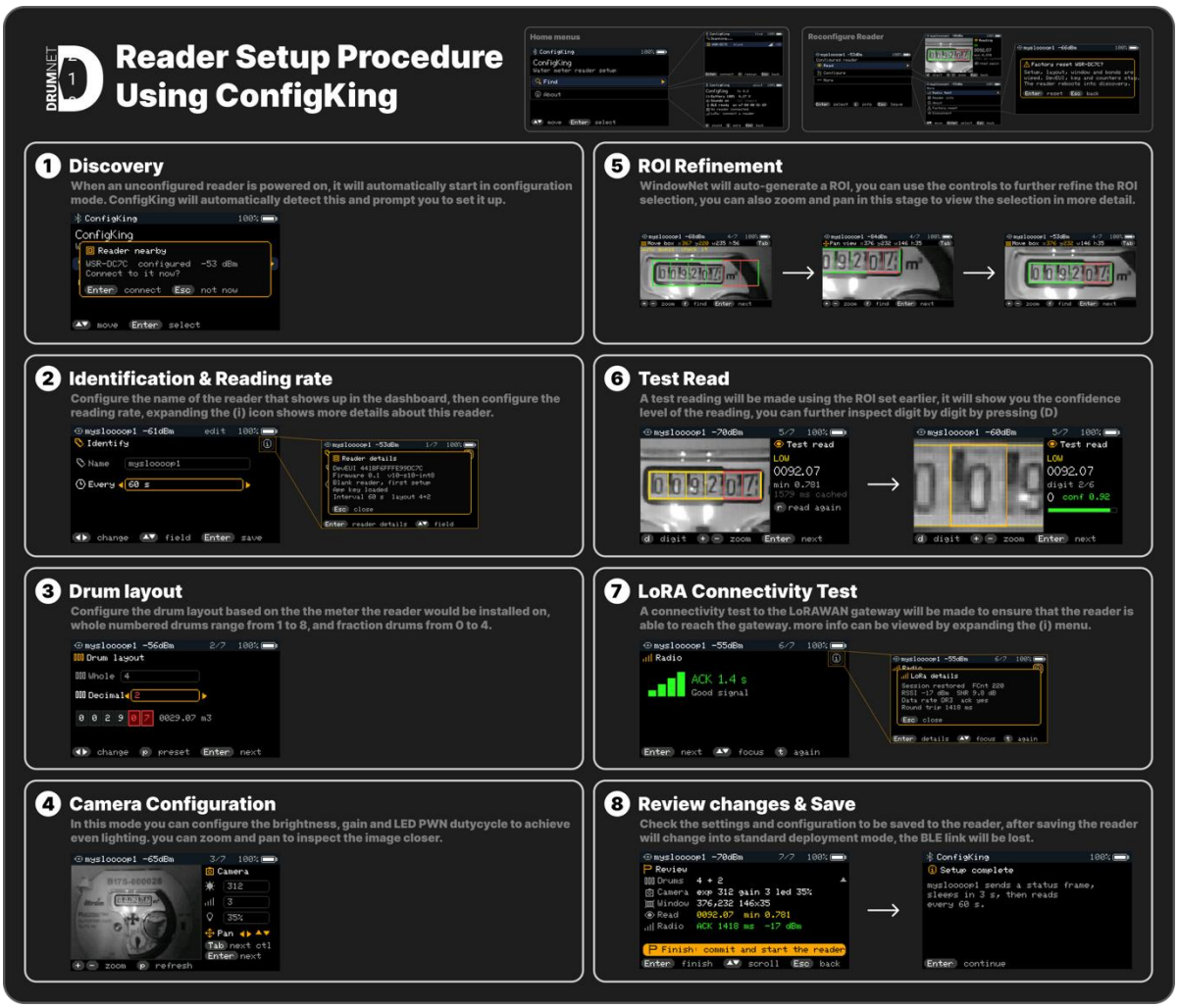


Figure 3.33 – Reader setup procedure using ConfigKing

4. Conclusion

Due to time constraints, I will need to stop the project as it currently is, there are still many issues that need to be addressed before this could be a viable solution.

4.1 Energy Use and Longevity

The energy use of this meter has not been measured or tested for its rated 2-year lifecycle, currently as it stands with Prototype V2, I do not think it is possible to achieve this 2-year lifecycle rating. A 3rd revision will be required to implement more energy saving components before it would be reasonable to start monitoring energy consumption of the reader. As the current firmware stands, it is close to my targeted level of efficiency and will require lesser work to reach an acceptable level.

4.2 WindowNet and Accuracy

The 56% reading accuracy of DrumNet V10 is a far cry from the targeted 99% accuracy the goal of this project stated. The primary bottleneck of accuracy is WindowNet, most erroneous readings were caused by WindowNet's inability to correctly find the correct ROI, resulting in the readers receiving bad data. Considerable work still needs to be put in to improve WindowNet to a level of high accuracy.

On that note, the DrumNet readers are still displaying signs of under training, from the earlier results, showing the DrumNet readers a meter it has not seen before yields a low confidence rating. The readers are trained on a moderately large sized dataset, however that dataset was largely dominated by the 9 meters that SimGen2 could generate, therefore the variation between the data used to train DrumNet was very small. A more diverse and larger dataset could help improve both WindowNet and the DrumNet's accuracy and performance.

4.3 Shell and Mounting

The current shell design is much too bulky for a realistic deployment in the real world, a fundamental redesign and approach needs to be taken to the shell, pcb, and mount. The camera also needs to be replaced as it's the primary cause of the high Z-height of this design, a camera that is shorter and better at macro imaging will need to be selected. Weather sealing will also require considering different materials and designs.

4.4 Closing Remarks

Much of this project was my first attempt at something new, I am not exactly satisfied with the outcome of the project, but I am still proud of the knowledge and experience I have gained from embarking on this project. I am grateful for having the rare opportunity of time, resource and support I need to learn and experiment in the way that I do. Without which this project would not have been possible.

References

Alif Semiconductor. (n.d.). *Ensemble E3 series*. Retrieved October 29, 2026, from https://alifsemi.com/ensemble-e3-series/

Ambiq. (n.d.). *Artasie AM1805*. Retrieved October 29, 2026, from https://ambiq.com/artasie-am1805

Espressif Systems. (n.d.). *ESP-NN: Optimised neural network functions for Espressif chipsets* [Computer software]. GitHub. Retrieved October 29, 2026, from https://github.com/espressif/esp-nn

Espressif Systems. (2020, December 31). *Announcing ESP32-S3 for AIoT applications* [Press release]. https://www.espressif.com/en/news/ESP32_S3

Espressif Systems. (2026a, May 8). *ESP32-P4 chip upgrade: Performance and multimedia experience enhancement* [Press release]. https://www.espressif.com/en/news/ESP32_P4_v3.x_Upgrade

Espressif Systems. (2026b). *ESP32-P4 series datasheet* (Pre-release Version 0.7). https://documentation.espressif.com/esp32-p4_datasheet_en.pdf

Espressif Systems. (2026c). *ESP32-S3 series datasheet* (Version 2.2). https://documentation.espressif.com/esp32-s3_datasheet_en.pdf

Espressif Systems. (2026d, July 27). *ESP32-S31 now in mass production and available for purchase* [Press release]. https://www.espressif.com/en/news/ESP32_S31_Mass_Production

Espressif Systems. (2026e). *ESP32-S31 series datasheet* (Pre-release Version 0.5). https://documentation.espressif.com/esp32-s31_datasheet_en.pdf

JLCPCB. (n.d.). *PCB prototype & PCB fabrication manufacturer*. Retrieved October 29, 2026, from https://jlcpcb.com/

jomjol. (n.d.). *AI-on-the-edge-device* [Computer software]. GitHub. Retrieved October 14, 2026, from https://github.com/jomjol/AI-on-the-edge-device

KiCad. (n.d.). *KiCad* [Computer software]. Retrieved October 29, 2026, from https://www.kicad.org/

LoRa Alliance. (n.d.). *About LoRaWAN®*. Retrieved October 29, 2026, from https://lora-alliance.org/about-lorawan/

M5Stack. (n.d.). *Cardputer-Adv*. M5Stack Docs. Retrieved October 29, 2026, from https://docs.m5stack.com/en/core/Cardputer-Adv

Ministry of Sustainability and the Environment. (2022, July 5). *Written reply to parliamentary question on smart meters by Ms Grace Fu, Minister for Sustainability and the Environment*. https://www.mse.gov.sg/latest-news/written-reply-to-pq-on-smart-meters/

OmniVision Technologies. (2006). *OV2640 color CMOS UXGA (2.0 megapixel) CameraChip with OmniPixel2 technology* (Version 1.6) [Preliminary datasheet]. https://files.waveshare.com/wiki/common/OV2640DS_en.pdf

Onshape. (n.d.). *Onshape* [Computer software]. Retrieved October 29, 2026, from https://www.onshape.com/en/

pingdotgg. (n.d.). *T3 Code* [Computer software]. GitHub. Retrieved October 29, 2026, from https://github.com/pingdotgg/t3code

Poon, Y. X. (n.d.). *How Singapore is tapping innovation to save water*. GovInsider. Retrieved October 14, 2026, from https://govinsider.asia/intl-en/article/ridzuan-ismail-pub-singapore-smart-water-meters

PUB, Singapore's National Water Agency. (n.d.-a). *Meter reading guide: Residential customers* [Brochure]. Retrieved October 14, 2026, from https://www.pub.gov.sg/-/media/PUB/PDF/WSN/Meter-reading-guide-Residential-customers.pdf

PUB, Singapore's National Water Agency. (n.d.-b). *Smart water meter*. Retrieved October 14, 2026, from https://pub.gov.sg/Public/KeyInitiatives/Smart-Water-Meter

PUB, Singapore's National Water Agency. (2021, April 15). *PUB appoints SP Services to install Singapore's first 300,000 smart water meters* [Press release]. https://www.pub.gov.sg/Resources/News-Room/PressReleases/2021/04/PUB-appoints-SP-Services-to-install-Singapore

Saft. (2026). *LS 33600: Primary Li-SOCl2 cell, high energy density 3.6 V D-size bobbin cell* (Document No. 31007-2-0426) [Data sheet]. https://saft4u.saft.com/en/download_file/5241def1-8668-4a68-9d2e-820bd3c68589/English

Seeed Studio. (2024, October 12). *Wio-SX1262 introduction*. Seeed Studio Wiki. https://wiki.seeedstudio.com/wio_sx1262/

Smart Water Magazine. (2022, November 22). *Singapore's PUB to connect 300,000 Itron smart water meters*. https://smartwatermagazine.com/news/itron/singapores-pub-connect-300000-itron-smart-water-meters

Tailscale. (n.d.). *What is Tailscale?* Retrieved October 29, 2026, from https://tailscale.com/kb/1151/what-is-tailscale

Texas Instruments. (2020). *TPS63900 1.8-V to 5.5-V, 75-nA IQ buck-boost converter with input current limit and DVS* (SLVSET3D) [Data sheet]. https://www.ti.com/lit/ds/symlink/tps63900.pdf

Three. (2026). *Badger model 55 water-meter drum digits* (Version 2) [Data set]. Hugging Face. https://huggingface.co/datasets/S3CUR/badger-55-watermeter

Westin, S. H. (2010, April 21). *ISO 12233 test chart*. Cornell University. https://www.graphics.cornell.edu/~westin/misc/res-chart.html